PCSX-Redux

None

None

None

Table of contents

1. Home 4
2. PCSX-Redux menus 5
2.1 File 6
2.2 Emulation 7
2.3 Configuration 8
2.4 Debug 9
2.5 Help 10
2.6 GPU information 10
3. Compiling PCSX-Redux 11
3.1 Getting the sources 11
3.2 Windows 11
3.3 Linux 12
3.4 Compiling PSX code 13
4. Command Line Flags 15
4.1 Missing files 18
4.2 Where your data lives 19
5. Debugging 20
5.1 Debugging with PCSX-Redux 20
5.2 GDB server 21
5.3 Connecting Ghidra to PCSX-Redux 31
5.4 Misc Features 34
5.5 VRAM viewer 38
5.6 GPU Logger 39
6. Mips API 41
6.1 Description 41
6.2 Functions 42
7. Web server 45
7.1 Activation 45
7.2 REST API 45
8. SIO1 networking 49
8.1 Settings 49
8.2 Modes 50
8.3 Example 51
9. Lua 52
9.1 Introduction 52

9.2 Loaded libraries 54
9.3 Redux basic API 56
9.4 Rendering 69
9.5 File API 80
9.6 Building ISO images 90
9.7 PPF patches 99
9.8 Webserver Lua API 103
9.9 Memory and registers 104
9.10 Events 107
9.11 Breakpoints 111
9.12 Inline assembler 114
9.13 Handling of PSX binaries 116
9.14 Case studies 120
10. Openbios 0
10.1 Purposes of Openbios 0
10.2 Building 0
10.3 Status 0
10.4 Organization 0
10.5 Technicalities 0
10.6 Commentary 0
10.7 Legality 0

1. Home

Welcome to the PCSX-Redux emulator documentation.

You can get the emulator for various platforms here: https://github.com/grumpycoders/pcsx-redux#where

To discuss this emulator specifically, please join our Discord server:



To discuss PlayStation 1 development, hacking, and reverse engineering in general, please join the PSX.Dev Discord server:



Compiling PCSX-Redux
Menus
Command line arguments
Debugging with PCSX-Redux
Internal MIPS api
Web Server
Lua API
OpenBios

2. PCSX-Redux menus

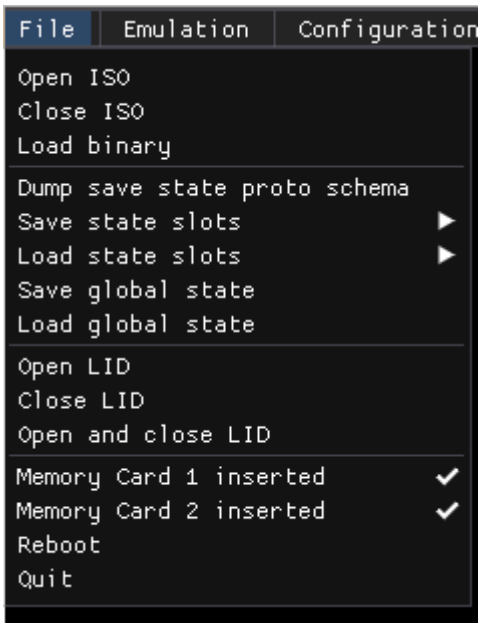
The menu bar holds some informations :



- CPU mode
- Game ID
- ImGui FPS counter (not psx internal fps)
- Guest FPS counter ("guest FPS") : how many times per second the emulated software moves its display start, which is how games flip buffers. Measured over one second of emulated vsyncs, 50 or 60 depending on the video setting. Reads 0 when the same buffer stays on screen, as with interlaced or single buffered games, and some pause screens.
- Audio buffer size, in milliseconds and frames

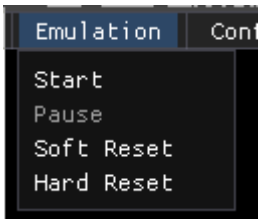
The FPS counters and the audio buffer size are only shown while the emulation is running; otherwise the menu bar displays "Idle".

2.1 File



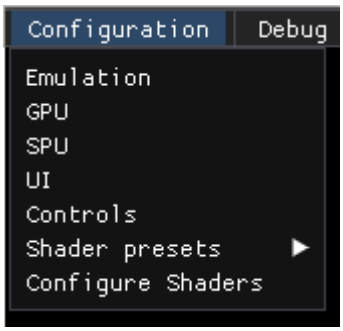
- Open Disk Image
- Reload Disk Image : Reopen the current disk image and hard reset
- Close Disk Image
- Load binary
- Add Lua archive
- Dump save state proto schema
- Save state slots
- Save global state
- Load state slots
- Load global state
- Open LID : Simulate open lid
- Close LID : Simulate closed lid
- Open and close LID : Simulate opening then closing the lid
- Reset settings...
- Reboot : Restart emulator
- Quit

2.2 Emulation



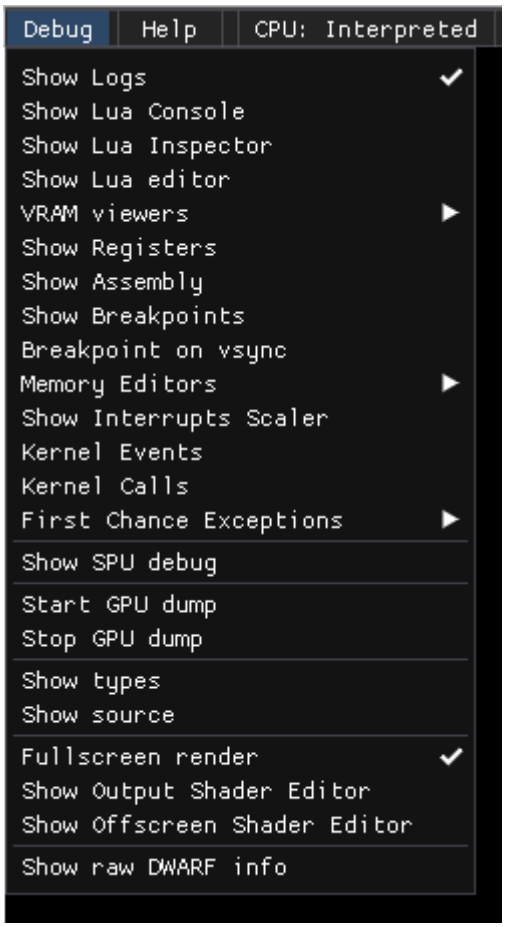
- Start emulation (F5): Start execution
- Pause emulation (F6): Pause execution
- Soft Reset (F8): Calls Redux's CPU reset function, which jumps to the BIOS entrypoint (0xBFC00000), resets some COP0 registers and the general purpose registers, and resets some IO. Does not clear vram.
- Hard Reset (Shift+F8): Similar to a reboot of the PSX.

2.3 Configuration



- Fullscreen
- Full window render
- Shader presets : Apply a shader preset
- Configure Shaders : Show shader editor
- Controls : Edit KB/Pad controls
- Manage Memory Cards : Open the memory card manager
- Emulation : Emulation settings
- GPU : Graphics Processing Unit settings
- SPU : Sound Processing Unit settings
- PIO Cartridge : Parallel port cartridge settings
- UI : Change user interface settings (such as font size, language or UI theme)
- System : System settings

2.4 Debug



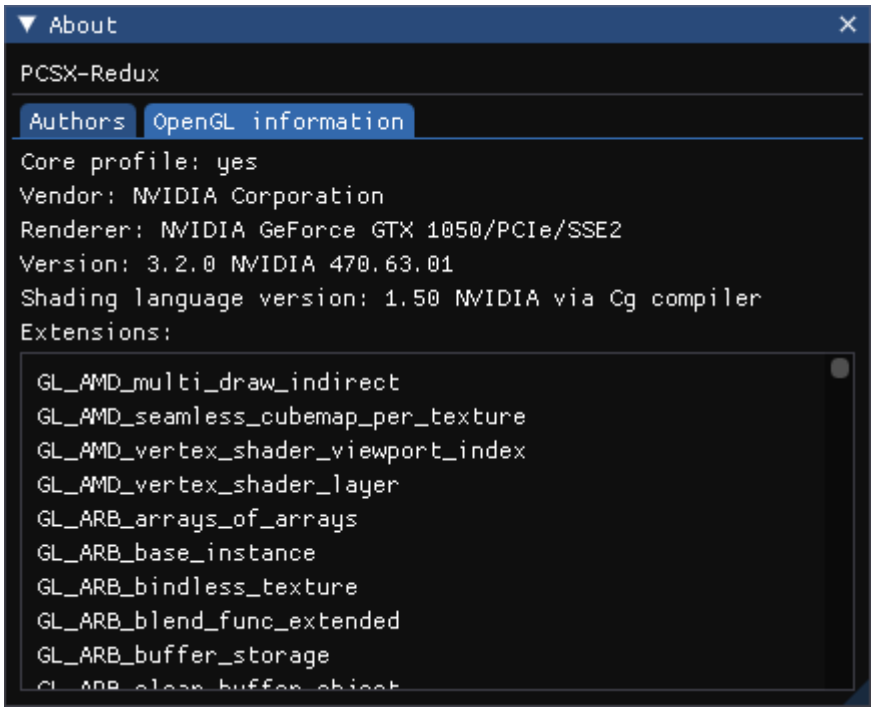
- Show Logs
- Lua : Lua console, inspector and editor
- CPU : Registers, assembly, breakpoints, callstacks, memory editors and other CPU debugging tools
- GPU : VRAM viewers, GPU logger and GPU debug
- SPU : SPU debug
- CD-Rom : Iso browser and CD-ROM viewer
- Misc hardware : Hardware registers and SIO1 debug
- Show PSYQo heap viewer
- Kernel : Kernel events, handlers and calls
- Rendering : Output and offscreen shader editors, and shader reset

2.5 Help

- Show ImGui Demo
- Show UvFile information
- About

2.6 GPU information

The 'About' dialog available in the 'Help' menu has an 'OpenGL information' tab that displays information on the GPU currently used by the program, such as the supported OpenGL extensions.



3. Compiling PCSX-Redux

3.1 Getting the sources

The only location for the source is on github. Clone recursively, as the project uses submodules:

`git clone https://github.com/grumpycoders/pcsx-redux.git --recursive`.

The PlayStation side of the code, which includes OpenBIOS, PSYQo, and the MIPS tests, lives in its own repository, nugget, which is mounted as a submodule at `src/mips`. Nugget has submodules of its own, so the clone needs to be recursive for it to be complete.

If you already have a checkout, run the following after pulling, so that new or moved submodules get fetched:

```
1   git submodule update --init --recursive
```

3.2 Windows

Install Visual Studio 2026 Community Edition using the `Desktop development with C++` workload, and the `C++ Clang tools for Windows` component, as some of the projects use the ClangCL toolset.
Open the file `vsprojects\pcsx-redux.sln`, select `pcsx-redux -> pcsx-redux`, right click, `Set as Startup Project`, and hit `F7` to build.
The project follows the open-and-build paradigm with no extra step, so no specific dependency ought to be needed, as NuGet will take care of downloading them automatically for you on the first build.

Note: If you get an error saying `hresult e_fail has been returned from a call to a com component`, you might need to delete the .suo file in vsproject/vs, restart Visual Studio and retry.

Openbios

Using Visual Studio Code, one can use the task "make_openbios" to compile: CTRL-P then `task make_openbios` to compile.

3.3 Linux

3.3.1 Compiling with Docker

Run `./dockermake.sh`. You need docker for this to work.

```
# Debian derivative; Ubuntu, Mint...
sudo apt install docker.io
# Arch derivative; Manjaro...
sudo pacman -S docker
```

You will also need a few libraries on your system for this to work. Check the Dockerfile for a list of library packages to install.

3.3.2 Compiling with make

- Debian derivatives (for full emulator compilation):

```
sudo apt-get install -y build-essential git make pkg-config clang g++ libcapstone-dev libfreetype-dev libavcodec-dev libavformat-dev libavutil-dev libcurl4-openssl-dev libsdl3-dev libswresample-dev libuv1-dev zlib1g-dev
```

- Arch derivatives :

```
sudo pacman -S capstone clang git make pkg-config ffmpeg freetype2 libuv zlib sdl3 curl xorg-server-xvfb
```

You can then just enter the 'pcsx-redux' directory and compile without using docker with `make`.

If you have a different mips compiler, you'll need to override the `PREFIX` and `FORMAT` variables, which default to `PREFIX=mipsel-none-elf FORMAT=elf32-littlemips`.

Openbios

Building OpenBIOS on Linux can be done with docker : `./dockermake.sh openbios`, or using `make`, with a `mipsel-none-elf` toolchain installed (see below) ; `make openbios`.

3.3.3 MacOS

You need MacOS Catalina with the latest XCode to build, as well as a few homebrew packages.
Run the brew installation script to get all the necessary dependencies.

Run `make` to build.

Compiling OpenBIOS will require a mips compiler, that you can generate using the following commands:

Openbios

```
brew install nikitabobko/tap/brew-install-path
brew install-path ./tools/macos-mips/mipsel-none-elf-binutils.rb
brew install-path ./tools/macos-mips/mipsel-none-elf-gcc.rb
```

Then, you can compile OpenBIOS using `make -C ./src/mips/openbios`.

3.4 Compiling PSX code

If you're only interested in compiling psx code, you can clone the PCSX-Redux repo;

```
git clone https://github.com/grumpycoders/pcsx-redux.git --recursive
```

then install a mips toolchain and get the converted PsyQ libraries in the `pcsx-redux/src/mips/psyq/` folder as per these instructions.
The `src/mips` folder is the nugget submodule, so it is only populated if the clone was recursive.

You can also find the pre-compiled converted Psyq libraries online.

3.4.1 Getting the toolchain on Windows

Install the `mips` toolchain manager script by copy-pasting the following into a command prompt:

```
powershell -c "& { iwr -UseBasicParsing https://raw.githubusercontent.com/grumpycoders/pcsx-redux/main/mips.ps1 | iex }"
```

Then, open a new command prompt, and install the toolchain, which will also be added to your `PATH`:

```
mips install 16.2.0
```

You can test it's working by launching a command prompt and typing `mipsel-none-elf-gcc.exe --version`. If you get a message like `mipsel-none-elf-gcc (GCC) 16.2.0`, then it's working !

3.4.2 Getting the toolchain on GNU/Linux

Debian derivative; Ubuntu, Mint...

There's no distribution package for the `mipsel-none-elf` toolchain, so build it from source, from the root of the repository:

```
sudo apt-get install -y make wget bzip2 xz-utils bison flex texinfo libgmp-dev
libmpfr-dev libmpc-dev
sudo bash tools/linux-mips/spawn-compiler.sh
```

Arch derivative; Manjaro...

The mipsel environment can be installed from AUR : mipsel-none-elf-binutils and mipsel-none-elf-gcc using your AURhelper of choice:

```
trizen -S mipsel-none-elf-binutils mipsel-none-elf-gcc
```

4. Command Line Flags

You can launch `pcsx-redux` with the following command line parameters:

The parsing code doesn't care about the number of dashes in the parameter's flag, so '-' can be used as well as '--', or any number of dashes.

Flag	Meaning
`-dumpproto`	Dump the protobuf schemas for PCSX-Redux on stdout and exit immediately.
`-version`	Print the version information as JSON on stdout and exit immediately.
`-no-ui`	Run without the graphical user interface, using a text interface instead. Implies `-stdout` and `-lua_stdout`.
`-cli`	Same as `-no-ui`, and also implies `-safe`.
`-run`	Begin execution immediately on startup.
`-stdout`	Redirect log output to stdout.
`-lua_stdout`	Redirect Lua's console output to stdout.
`-logfile`	Specify a file to log output to.
`-bios`	Specify a BIOS file.
`-testmode`	Interpret internal API's `pcsx_exit()` command as a request to exit the emulator instead of pausing, and close the emulator. Implies `-safe`, `-no-gui-log`, and will also disable first chance exceptions. If no BIOS could be loaded, the emulator exits with code -1 (255 on POSIX systems) right after the "No BIOS loaded" message, instead of halting. Use only when doing unit testing.
`-exe`	Load a PSX exe.
`-loadexe`	Load a PSX exe.
`-iso`	Load a PSX disk image (iso, bin/cue).
`-loadiso`	Load a PSX disk image (iso, bin/cue).
`-disk`	Load a PSX disk image (iso, bin/cue).
`-memcard1`	Specify a memory card file to use as memory card slot 1.
`-memcard2`	Specify a memory card file to use as memory card slot 2.
`-pcdrv`	Enable the pcdrv device interface. (Access PC filesystem through SIO).
`-no-pcdrv`	Disable the pcdrv device interface. Will change the saved setting.
`-pcdrvbase`	Specify base directory for pcdrv.
`-safe`	Resets configuration to defaults.
`-resetui`	Resets the UI to its defaults.
`-noshaders`	Displays the emulated output without going through the output shaders.
`-noupdate`	Disables the automatic update checks.
`-viewports`	Enables ImGui viewports, allowing windows to be dragged outside of the main window. Enabled by default except on Linux.
`-no-viewports`	Disables ImGui viewports.
`-kiosk`	Enables kiosk mode, disabling UI interaction. Will change the saved setting.
`-no-kiosk`	Disables kiosk mode, allowing the user to interact with the UI. Will change the saved setting.
`-interpreter`	Use the interpreter CPU core.
`-dynarec`	Use the dynamic recompiler CPU core.

Flag	Meaning
`-openglgpu`	Use the OpenGL GPU renderer. Will change the saved setting.
`-softgpu`	Use the software GPU renderer. Will change the saved setting.
`-8mb`	Emulates 8MB of RAM instead of 2MB. Will change the saved setting.
`-2mb`	Emulates 2MB of RAM. Will change the saved setting.
`-debugger`	Activates the debugger. Breakpoints, whether set from the UI, Lua or GDB, only trigger while it is active. Will change the saved setting.
`-no-debugger`	Deactivates the debugger. Will change the saved setting.
`-fastboot`	Skips the BIOS logo and boot animation. Will change the saved setting.
`-no-fastboot`	Shows the BIOS logo and boot animation. Will change the saved setting.
`-gdb`	Activates the gdb server. Will change the saved setting.
`-no-gdb`	Deactivates the gdb server. Will change the saved setting.
`-gdb-port`	Sets the TCP port the gdb server is listening on. Will change the saved setting.
`-webserver`	Activates the web server. Will change the saved setting.
`-no-webserver`	Deactivates the web server. Will change the saved setting.
`-webserver-port`	Sets the TCP port the web server is listening on. Will change the saved setting.
`-trace`	Activates the CPU trace logging. Will change the saved setting.
`-no-trace`	Deactivates the CPU trace logging. Will change the saved setting.
`-no-gui-log`	Fully disables logs to be sent to the GUI.
`-archive`	Specifies a .zip file to load for the `Support.extra.dofile` function.
`-dofile`	Specifies a Lua file to load through the `Support.extra.dofile` function.
`-exec`	Specifies a Lua string to execute.
`-luacov`	Enables Lua code coverage report. Requires the `luacov` Lua module to be installed.
`-portable`	Enables portable mode. Settings and saves are stored in the current directory, or in the directory given as an optional argument to this flag. See Where your data lives.
`-no-portable`	Disables portable mode, overriding any of the automatic detections described below.

4.1 Missing files

Before anything starts, the files given to `-bios`, `-iso`, `-loadiso`, `-disk`, `-exe`, `-loadexe` and `-archive` must exist, and `-pcdrvbase` must be a directory. Otherwise the emulator prints one line per missing path on stderr and exits with code 1. A missing `-bios` file is an error: the OpenBIOS fallback only applies to a BIOS set in the configuration.

`-memcard1` and `-memcard2` are not checked, since missing memory cards are created on demand. Neither is `-dofile`, which is looked up through `-archive`.

4.2 Where your data lives

Settings, memory cards and save states go in one of two places.

Normally they go in a per-user directory: `%APPDATA%\pcsx-redux` on Windows and `$HOME/.config/pcsx-redux` everywhere else, macOS included. It is created on first run.

In portable mode they go in the portable directory instead, which is the current directory unless something else set it. Portable mode turns on when any of these is true:

- `-portable` was passed;
- a `pcsx.json` exists in the current directory;
- a `pcsx.json` exists next to the executable, in which case the portable directory is the executable's directory and not the current one;
- the emulator is being run out of its own source tree.

`-no-portable` turns it off again.

The current directory is not always the directory holding the binary. A shortcut with its own start-in, a file association on a disc image, or a frontend will each hand the emulator something else. For a portable install that follows the binary, put the `pcsx.json` next to it; then the launch directory stops mattering.

5. Debugging

5.1 Debugging with PCSX-Redux

PCSX-Redux has strong debugging capabilities. It has a built-in GDB server, which allows you to connect to it with a GDB client, such as gdb itself when targeting MIPS, a vscode connector, IDA Pro, or Ghidra, and debug the MIPS CPU. See debugging with Ghidra for more information on debugging with Ghidra.

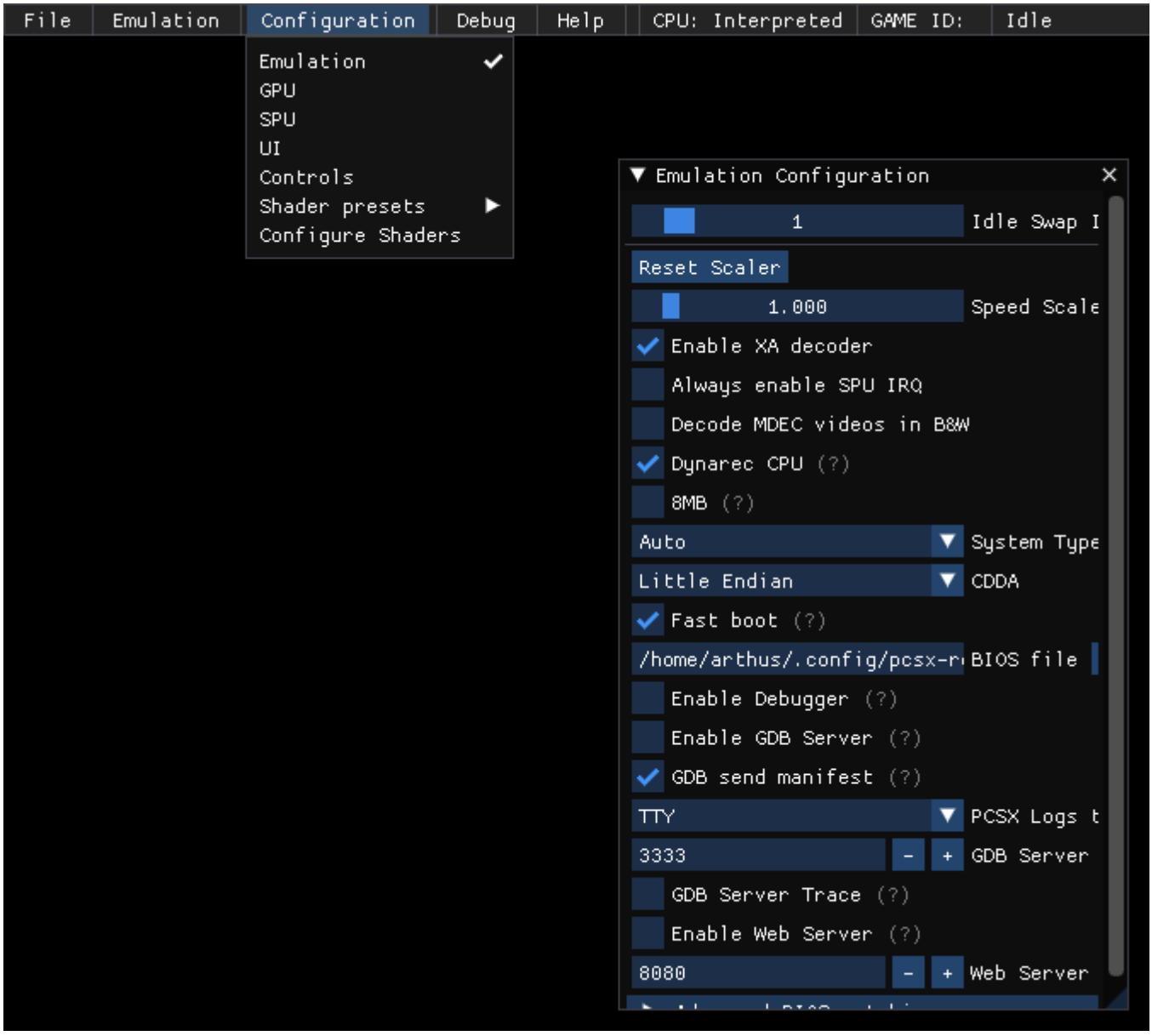
There are also built-in debugging tools, available in the Debug menu. Most of the CPU debugging features will require switching the Dynarec off from the Emulation configuration menu, as the Dynarec is not compatible with the debugging features. Additionally, the debugger needs to be enabled, also in the Emulation configuration menu.

The GPU debugging tools can work with the Dynarec enabled, and thus will be much faster than when the interpreter is used.

5.2 GDB server

The GDB server allows you to set breakpoints and control your PSX program's execution from your gdb compatible IDE.

5.2.1 Enabling the GDB server



In PCSX-Redux: `Configuration > Emulation > Enable GDB Server`.

Make sure the debugger is also enabled, either from the configuration menu or with the `-debugger` command line flag. Without it, the server refuses breakpoint and watchpoint requests (`Z` packets) with an error.



5.2.2 GDB setup

You need `gdb-multiarch` on your system :

Windows

Download a pre-compiled version from here : https://static.grumpycoder.net/pixel/gdb-multiarch-windows/

GNU/Linux

DEBIAN BASED

Install via your package manager :

```
# Debian derivative; Ubuntu, Mint...
sudo apt install gdb-multiarch
```

ARCH BASED

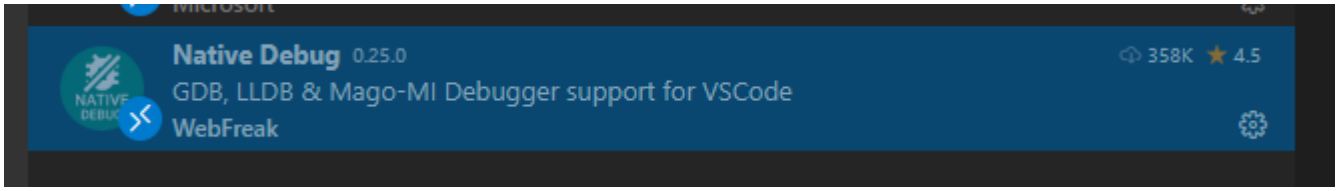
On Arch based distributions, multiarch is now enabled by default in regular builds and you don't need to install a specific version anymore.
You can install the 'gdb' package with `pacman` :

```
sudo pacman -S gdb
```

5.2.3 IDE setup

MS VScode

- Install the `Native debug` extension :
  https://marketplace.visualstudio.com/items?itemName=webfreak.debug



- Adapt your `launch.json` file to your environment :
  A sample `launch.json` file is available here.
  This should go in `your-project/.vscode/` .

You need to adapt the values of `"executable"` , `"gdbpath"` and `"autorun"` according to your system :

EXECUTABLE

This is the path to your `.elf` executable :

```
1       "executable": "HelloWorld.elf",
```

https://github.com/grumpycoders/pcsx-redux/blob/a3bebd490388130e924124cdfeff3bc46b6149d9/.vscode/launch.json#L153

GDBPATH

This the path to the `gdb-multiarch` executable:

```
1       "gdbpath": "/usr/bin/gdb-multiarch",
```

https://github.com/grumpycoders/pcsx-redux/blob/a3bebd490388130e924124cdfeff3bc46b6149d9/.vscode/launch.json#L154-L157

AUTORUN

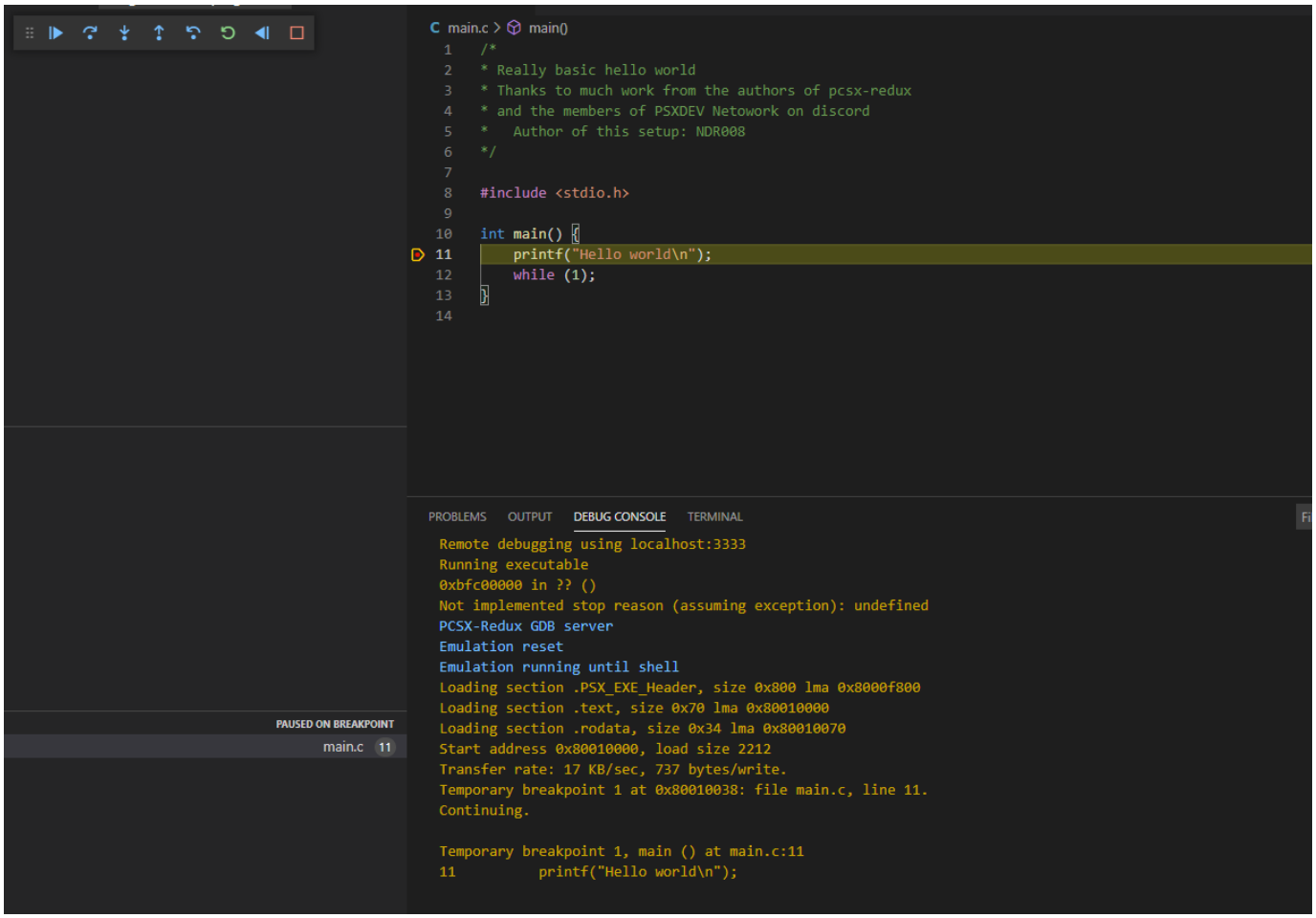
```
    "autorun": [
     "monitor reset shellhalt",
     [...]
     "load your-file.elf",
```

Make sure that `"load your-file.elf"` corresponds to the `"executable"` value.

https://github.com/grumpycoders/pcsx-redux/blob/a3bebd490388130e924124cdfeff3bc46b6149d9/.vscode/launch.json#L159-L165

By default, using `localhost` should work, but if encountering trouble, try using your computer's local IP (e.g; 192.168.x.x, 10.0.x.x, etc.)

https://github.com/grumpycoders/pcsx-redux/blob/a3bebd490388130e924124cdfeff3bc46b6149d9/.vscode/launch.json#L150

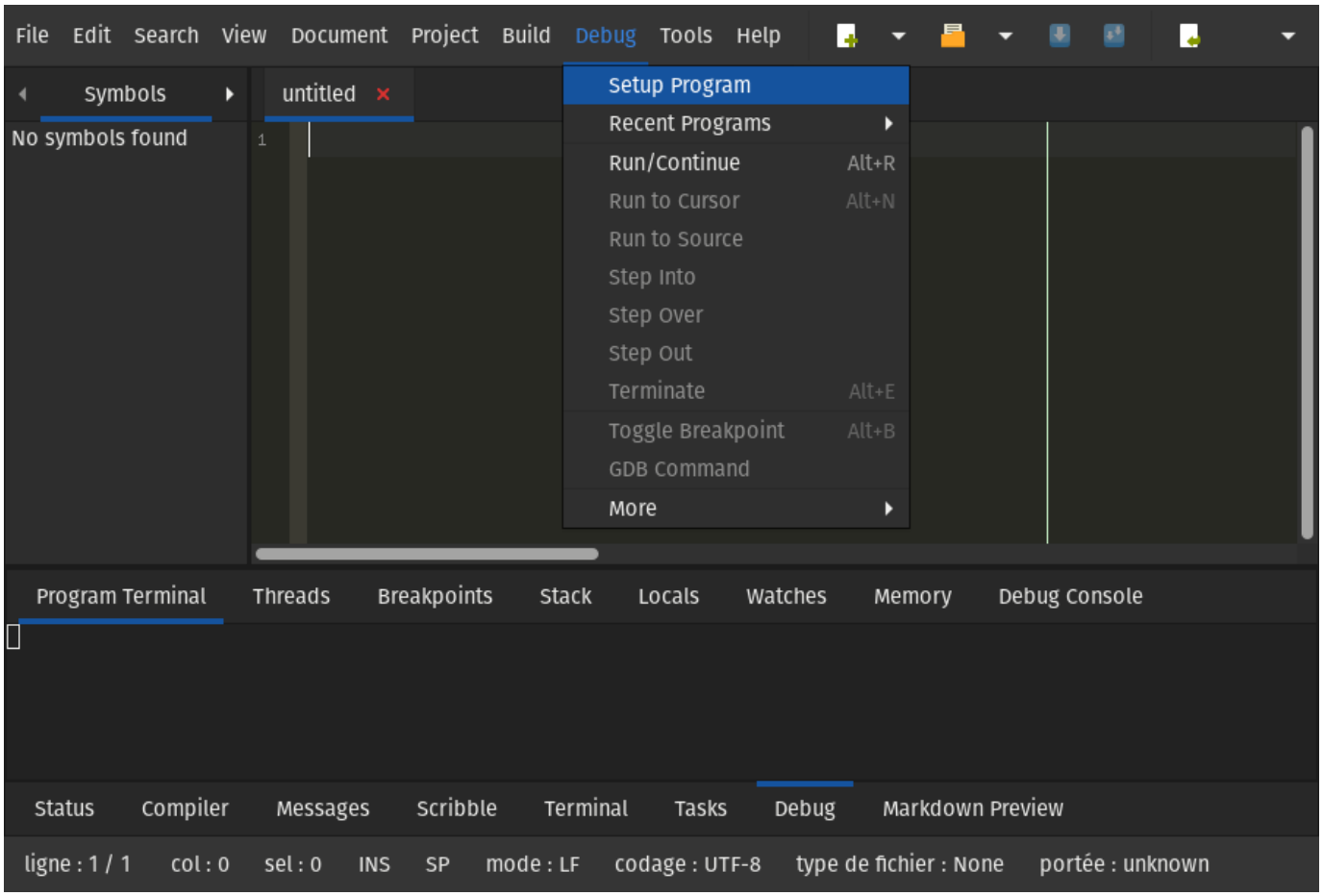


Geany

Make sure you installed the official plugins and enable the `Scope debugger`.

To enable the plugin, open Geany, go to `Tools > Plugin manager` and enable `Scope Debugger`.

You can find the debugging facilities in the `Debug` menu ;



You can find the plugin's documentation here : https://plugins.geany.org/scope.html

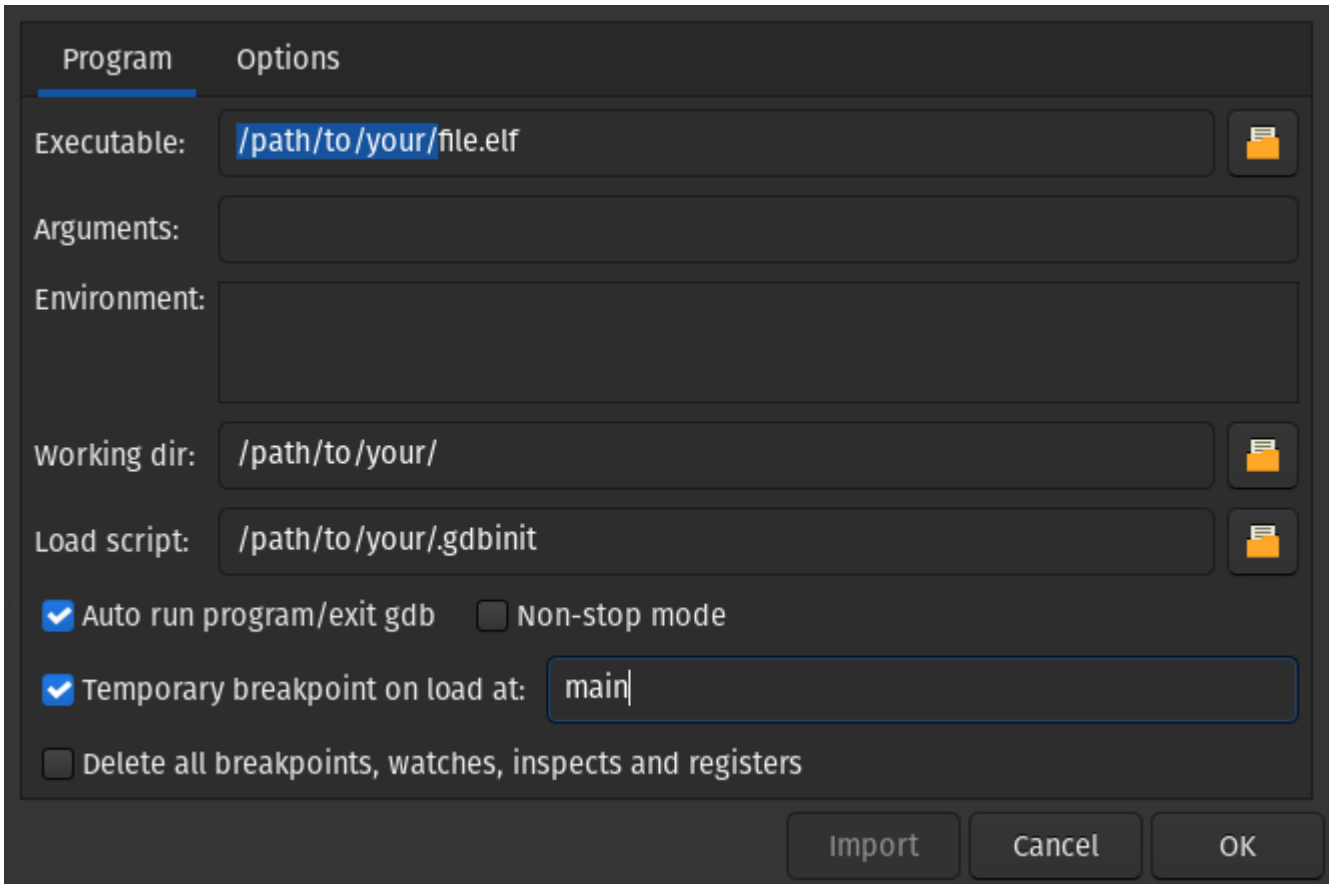
.GDBINIT

Create a `.gdbinit` file at the root of your project with the following content, adapting the path to your `elf` file and the gdb server's ip.

```
target remote localhost:3333
symbol-file /path/to/your/executable.elf
monitor reset shellhalt
load /path/to/your/executable.elf
```

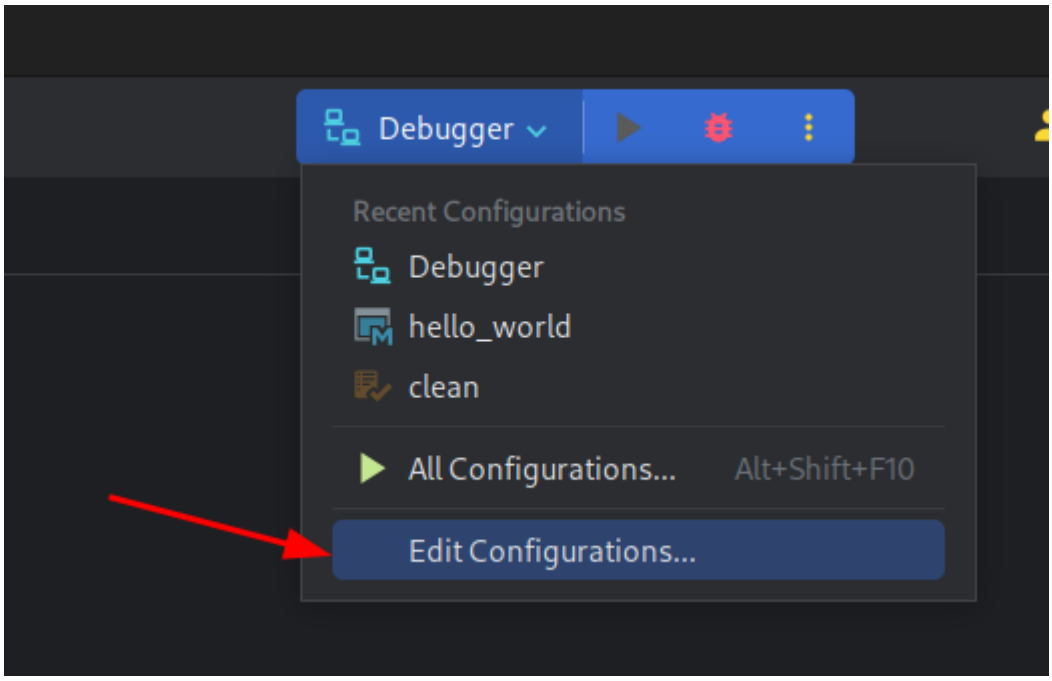
PLUGIN CONFIGURATION

In Geany : `Debug > Setup Program` :

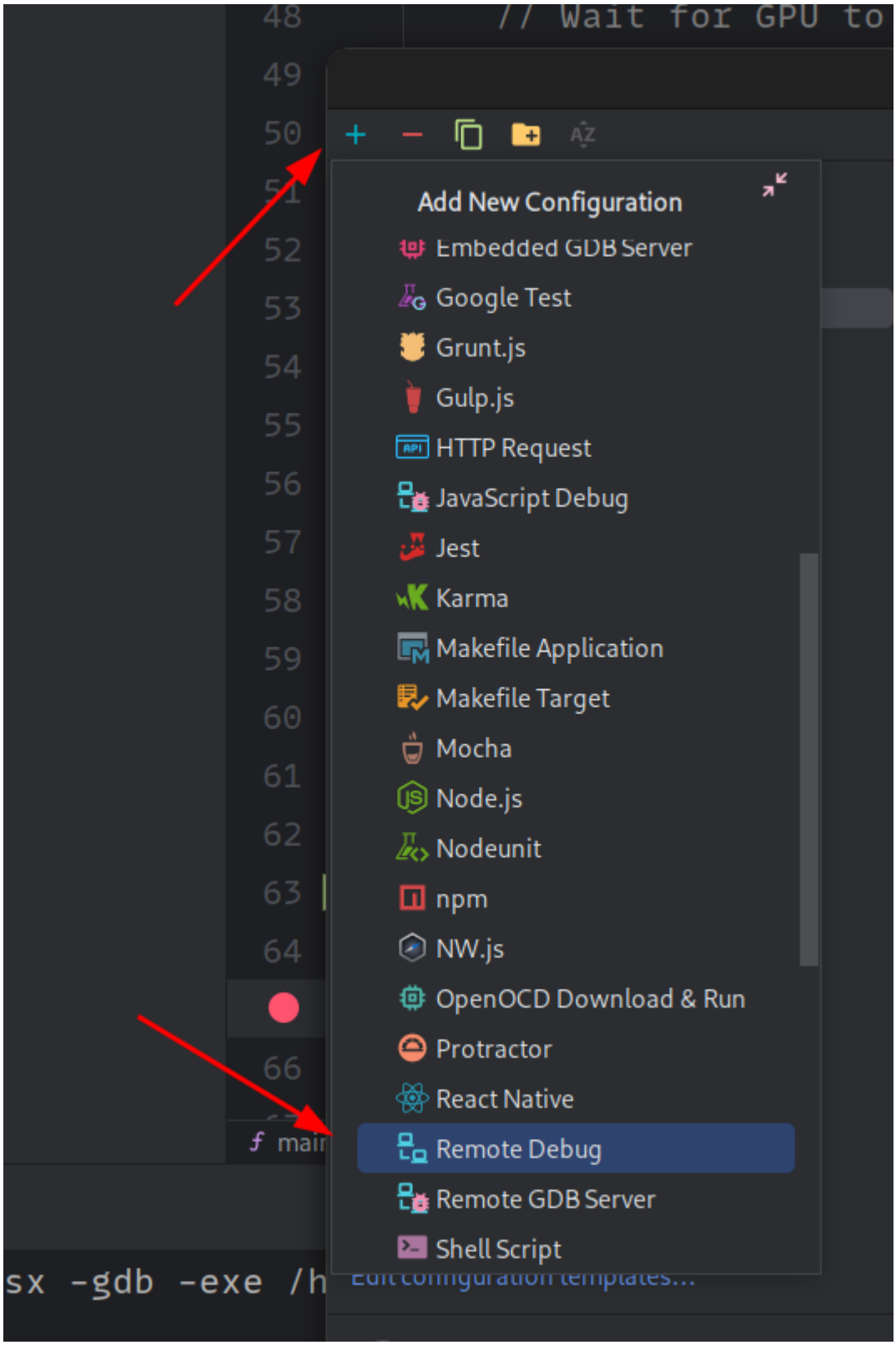


CLion

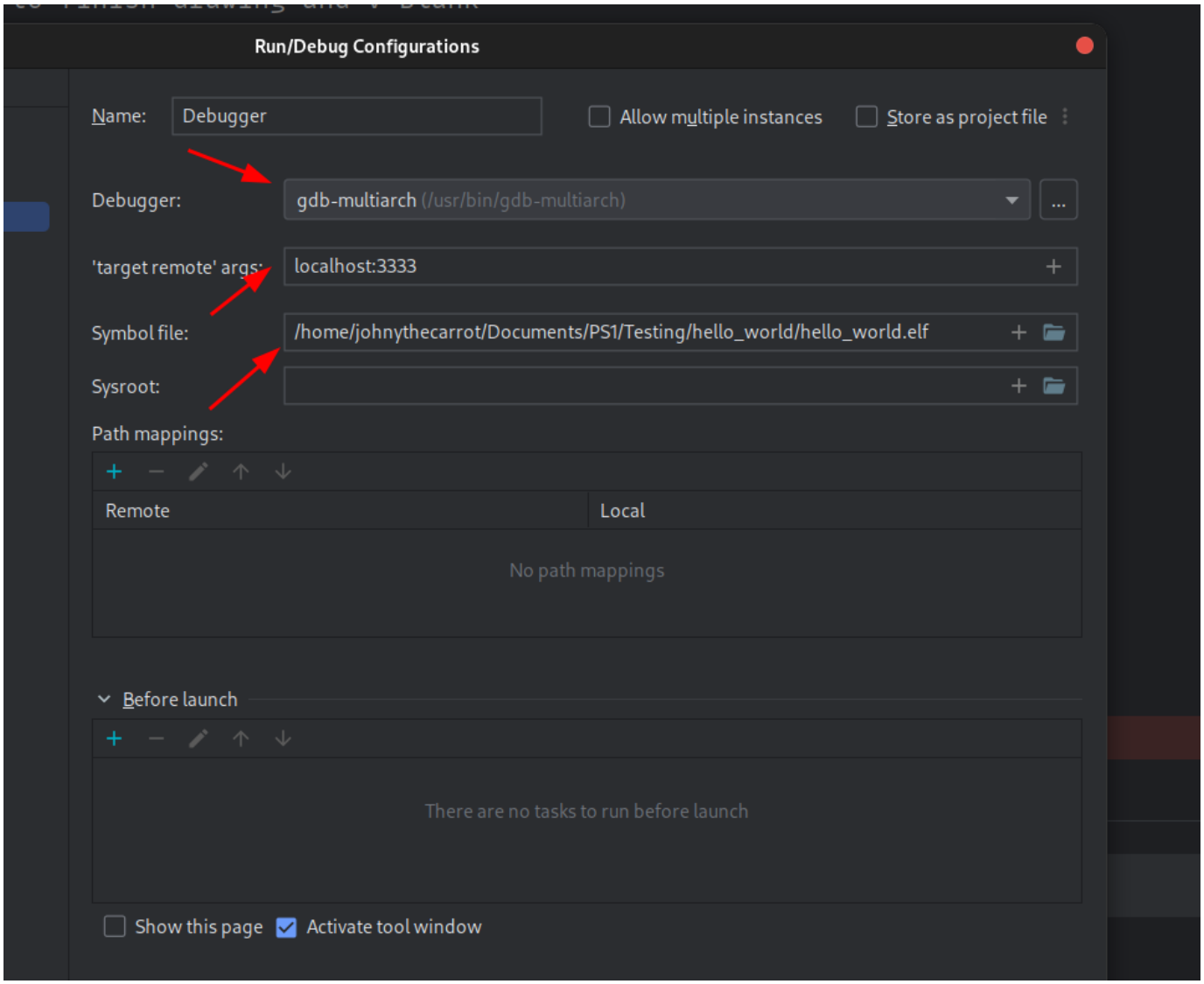
Open the Run/Debug Configurations menu, which you can find here:



Then, add a new Remote Debug configuration:



Finally, set your new configuration up:



.GDBINIT

Create a `.gdbinit` file at the root of your project with the following content, adapting the path to your `elf` file.

```
define target remote
target extended-remote $arg0
symbol-file /path/to/your/executable.elf
monitor reset shellhalt
load /path/to/your/executable.elf
end
```

5.2.4 Beginning Debugging

Launch `pcsx-redux`, then run the debugger from your IDE. It should load the `elf` file, and execute until the next breakpoint.

Starting debugging in Geany

Your browser does not support the video tag.

Source：
https://archive.org/details/pcsx_redux_geany_gdb

5.2.5 Watchpoints

Write, read and access watchpoints (`watch`, `rwatch` and `awatch` in gdb) cover the full length gdb requests, so watching a whole struct or array stops on an access anywhere in it. A watchpoint of length 0, or one wrapping around the end of the address space, is refused with an error.

5.2.6 Additional tools

https://github.com/cyrus-and/gdb-dashboard/

5.3 Connecting Ghidra to PCSX-Redux

Since version 10.3, Ghidra now supports debugging MIPS targets. This allows for a much more powerful reverse engineering experience than what was previously possible with the GDB server. This document will explain how to set up Ghidra to debug PCSX-Redux, as it is not entirely straightforward.

5.3.1 Prerequisites

- A gdb "multiarch" binary is required. For Windows, you can get it from here. For Linux, you can get it from your distribution's package manager; on Ubuntu and Debian, this is the package `gdb-multiarch`. And for MacOS, you can use the brew package manager to install it; this is the package `gdb`.
- Ghidra 10.3 or newer. You can get it from here.
- PCSX-Redux either configured to disable Dynarec, enable the debugger, and enable the gdb server, or started using the following command-line arguments: `-interpreter -debugger -gdb`.
- The following file downloaded somewhere on your computer, naming it `ghidra_debugger_scripts`.

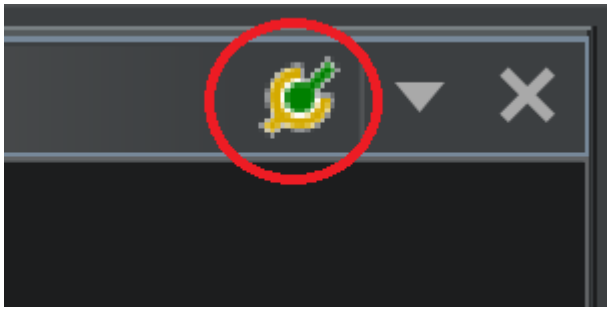
5.3.2 Setting up Ghidra

Before starting Ghidra, until version 10.3.3, the MIPS CPU isn't terribly well defined. One needs to go to the installation files of Ghidra, and edit the file `Ghidra/Processors/MIPS/data/languages/mips.ldef`. In this file, find the lines `<external_name tool="gnu" name="mips:4000"/>`, and change them to `<external_name tool="gnu" name="mips:3000"/>`. This will allow Ghidra to properly recognize the MIPS CPU used by the PlayStation 1. This step is no longer necessary starting with Ghidra 10.3.3.

5.3.3 Setting up Ghidra's debugger

When in the main view of Ghidra, right click on the project you want to debug, and in the context menu, select `Open With > Debugger`. This will open the debugger tool instead of the default disassembler tool.

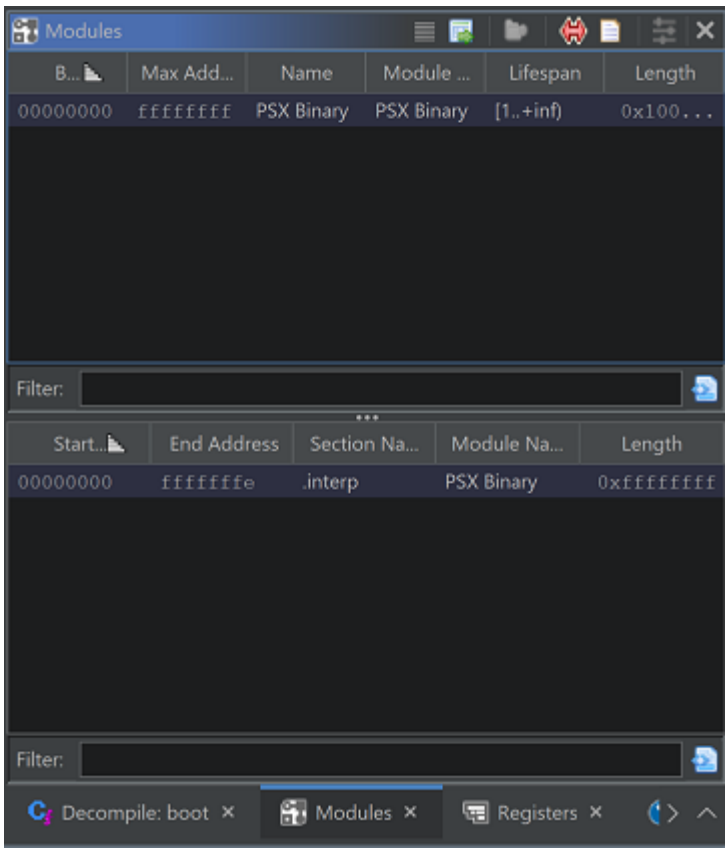
First, identify the Debugger Targets window, and click its top right button:



This will open the debugger connector window. In the drop down, select `gdb`, and as the launch command, enter the path to the gdb multiarch binary, followed by `-i mi2`. For example, on Windows, this could be `C:/gdb-multiarch/bin/gdb-multiarch.exe -i mi2`. Click `Connect`.

A new `Interpreter` window should open on the right, with the prompt `(gdb)` allowing you to type commands. First, you need to source the `ghidra_debugger_scripts` file from before. To do this, type `source <path to ghidra_debugger_scripts>`. For example, on Windows, this could be `source C:/Users/Pixel/Downloads/ghidra_debugger_scripts`. Then, you need to connect to the PCSX-Redux gdb server. To do this, type `target remote localhost:3333`. Finally, locate the `Modules` tab in

the right window, next to the `Interpreter` tab, which should look like this:



Select the top line, right click on it, and in the context menu, select `Map Module to <name of your project>`. In the new window that appears, simply click `Ok`.

At this point, Ghidra should be fully connected to PCSX-Redux, and should be able to place breakpoint, resume or pause execution, inspect variables, etc. Please be aware that, as of Ghidra 10.3, many features of the debugger are still work in progress, and won't necessarily be stable.

5.4 Misc Features

5.4.1 Mapping breakpoints

PCSX-Redux has a feature that allows mapping the memory of the console while the software is running, and to set breakpoints on the mapped memory. This can for instance help in finding codepath when performing certain activities when running code.

First, map the kind of action you want to discover, such as executing code, reading memory, or writing memory. Then, run the code for some time without performing the specific action you want to discover. Finally, activate the map breakpoint mode, and then perform the action you want to discover. The breakpoint should be triggered when the action is performed.

For example, say that in a game, you want to know what code is executed when you press the "X" button. First, check the `Map execution` checkbox. Then, run the game for a while without pressing the "X" button. This will map enough of the memory that's being run in a normal way. Finally, activate the `Break on execution map` checkbox, and press the "X" button. If the game takes a new codepath that hasn't been executed yet, the breakpoint should be triggered.

Breakpoints are always checked before mapping the memory, so it's safe to keep both checkboxes on at the same time.

Click the `Clear maps` button to zero out all of the maps, when starting anew.

5.4.2 ISO browser file viewer

In the ISO browser (`Debug > CD-Rom > Show Iso Browser`), right-clicking a file offers `View` , next to `Extract` , `Replace` and `Hex Edit` . It opens a `View - <file>` window with up to two tabs:

- `TIM` : only present when the file parses as a TIM image. Shows its bpp, size and VRAM position, with a `palette` slider when the CLUT holds more than one palette.
- `Raw image` : decodes the file as 1, 2, 4, 8, 16 or 24 bpp pixels, with `width` , `height (0 = fit)` and `header bytes` set by hand. 16 bpp is read as PlayStation 15-bit colour, 1 to 8 bpp as greyscale.

Both tabs have a `zoom` slider. The viewers are written in Lua and loaded from `resources/fileviewers.lua`; if that file is missing or fails to load, the `View` item does not appear.

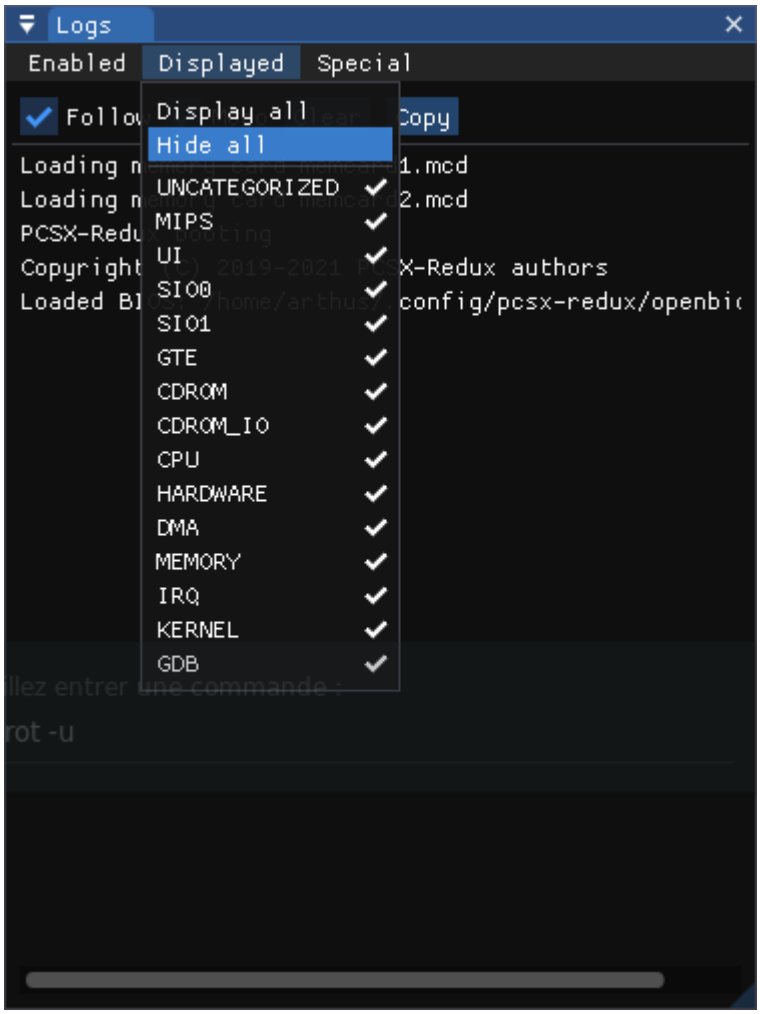
5.4.3 CPU trace dump

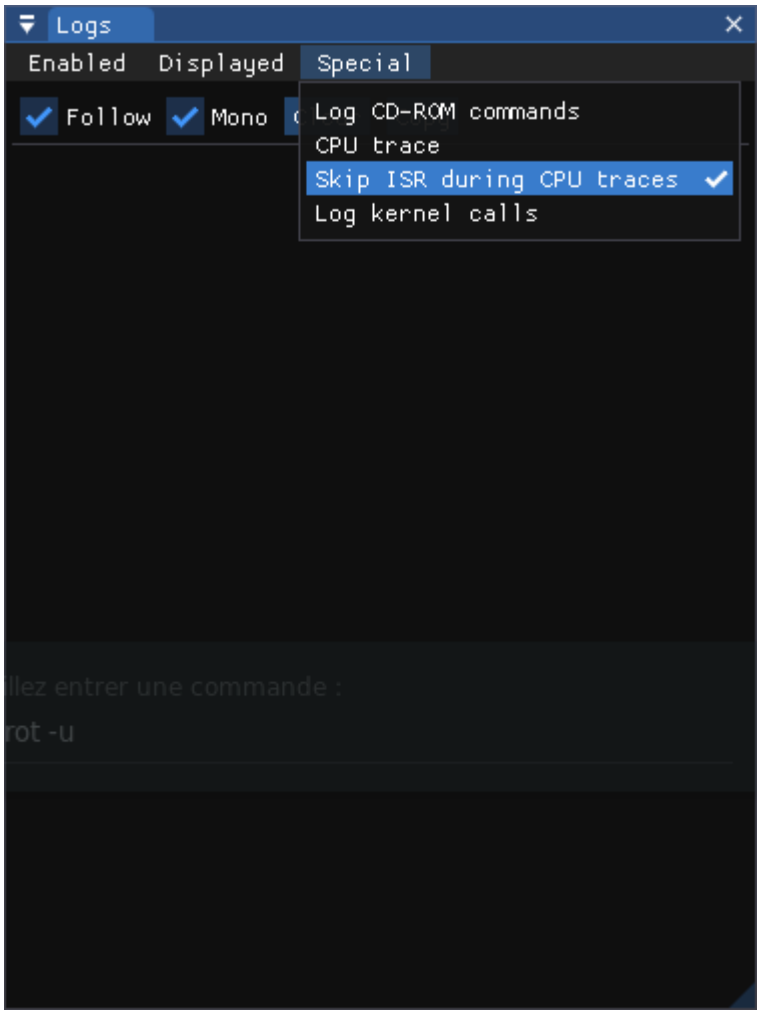
Setup

In PCSX-Redux, make sure `Debug > Show Logs` is enabled.

In the 'Logs' window, hide all logs : `Displayed > Hide all`

To avoid unnecessary noise, you can also skip ISR during CPU traces : `Special > Skip ISR during CPU traces`


Logs
Enabled
Displayed
Special
Display all
Hide all
UNCATEGORIZED
MIPS
UI
SIO0
SIO1
GTE
CDROM
CDROM_IO
CPU
HARDWARE
DMA
MEMORY
IRQ
KERNEL
GDB
Copy



Begin dump

To dump the CPU traces, launch pcsx-redux with the following command :

```
pcsx-redux -stdout -logfile log.txt
# Alternatively, you can use -stdout on its own and pipe the output to a file.
pcsx-redux -stdout >> log.txt
```

You can use additional flags to launch an executable/disk image in one go, e.g :

```
pcsx-redux -stdout -logfile tst.log -iso image.cue -run
```

Source

https://discord.com/channels/642647820683444236/663664210525290507/882608398993063997

5.5 VRAM viewer

5.5.1 Navigating

Holding the middle button, or both the left and right buttons, allows you to pan the view around. Using the wheel allows you to zoom in and out, at the location of the mouse cursor.

5.5.2 Lensing

Holding the CTRL key of your keyboard will bring up a lens, which will show you a locally zoomed version of the VRAM at the location of the mouse cursor. The lens zoom level can be changed by using the wheel while holding the CTRL key. Holding the CTRL and Shift buttons while using the wheel will change the size of the lens. The lens can be closed by releasing the CTRL key.

5.5.3 The various viewers

There are different viewers available from the main menu, which can be used to visualize the VRAM in different ways. The main viewer will let you see the VRAM using various CLUTs. The CLUT viewer will let you select a CLUT to use for the main VRAM viewer. In order to do this, first select the 8-bits or 4-bits view in the main viewer. Then, in the CLUT viewer, select `View -> Select a CLUT`. At this point, hovering the CLUT viewer will automatically change the main viewer to use the hovered CLUT. Once the proper view is found, simply click on the first pixel of the CLUT viewer to select the CLUT more permanently.

The GPU logger will also select CLUTs and change the main viewer's mode automatically, depending on the GPU commands being inspected.

5.6 GPU Logger

The GPU logger is a tool that allows you to see the GPU commands being executed by the emulator, and the resulting VRAM changes. It can be used to debug the GPU, and to understand how the executed software is rendering the scene. The logger will have the primitives sent since the last vsync, and will automatically clear the log when the first command after a new vsync is received. Note that if the PlayStation software isn't running at full FPS, the rendering of one of its frames may span over multiple vsyncs, and the logger will then only have the part sent since the last vsync.

Note that it can be fairly resource intensive, and may significantly slow down the emulation, depending on the context.

The top of the GPU Logger window will have the following checkboxes:

- GPU logging - Enable or disable the GPU logging.
- Breakpoint on vsync - Pause the emulation when a vsync occurs, allowing to inspect the current frame.
- Replay frame - Enables the replay of the current frame. See below for details.
- Show origins - Show the data path of the primitives. This will show the origin of the data, and the path it took to reach the GPU. For example, a sequence of primitives may be sent to the GPU via chained DMA.

5.6.1 Understanding the logs

The top of the logger can be expanded to display rough frame statistics. These values aren't necessarily too accurate, and are only meant to give a rough idea of the frame complexity.

Each row of the logger displays one command sent to the GPU. The first button and checkbox will be used for the replay system. The next three buttons and checkboxes will be used for the highlighting system. The next column will display the command name, and opening the tree node will expand the command parameters.

The expanded node may have buttons which will affect the main VRAM viewer, either by selecting CLUTs, or zooming in on the corresponding region. The VRAM viewer will also be updated when the replay system is used.

5.6.2 Highlighting Primitives

The GPU logger can highlight primitives in the VRAM viewer. One or more primitives may be selected, and the corresponding VRAM regions will be outlined. The highlighting will be cleared when a new frame is started. The default outlined colors will be red for written pixels, and green for read pixels. The colors can be changed in the main VRAM viewer settings.

Checking the `Highlight on hover` checkbox will temporarily outline a primitive when hovering it in the logger. This can be useful to quickly identify the corresponding primitive in the VRAM viewer by flicking the mouse over the logger.

Checking the second checkbox in a logger node will permanently highlight the corresponding primitive in the VRAM viewer. The `[B]` and `[E]` buttons will select the beginning and the end of a span of primitives, and highlight them in the VRAM viewer.

5.6.3 Replay System

Once a frame has been logged properly, and the emulator is paused, the replay system can be used to replay the frame. The replay system will constantly replay the frame as long as it is activated, and it will update the VRAM viewer accordingly. By default, all nodes in the logger will be selected for replaying. Unselecting the first checkbox in a node will prevent it from being replayed, and the VRAM viewer will show what happens when this primitive isn't executed, and potentially see what is underneath it. Clicking the `[T]` button of a node will select all nodes for replaying until this node, allowing to easily see the frame being built up to this point.

6. Mips API

6.1 Description

PCSX-Redux has a special API that mips binaries can use :

```
static __inline__ void pcsx_putc(int c) { *((volatile char* const)0x1f802080) = c; }
static __inline__ void pcsx_debugbreak() { *((volatile char* const)0x1f802081) = 0; }
static __inline__ void pcsx_execSlot(uint8_t slot) { *((volatile uint8_t*
const)0x1f802081) = slot; }
static __inline__ void pcsx_exit(int code) { *((volatile int16_t* const)0x1f802082)
= code; }
static __inline__ void pcsx_message(const char* msg) { *((volatile const char**
const)0x1f802084) = msg; }
static __inline__ void pcsx_checkKernel(int enable) { *((volatile char*)0x1f802088)
= enable; }
static __inline__ int pcsx_isCheckingKernel() { return *((volatile char*
const)0x1f802088) != 0; }

static __inline__ int pcsx_present() { return *((volatile uint32_t*
const)0x1f802080) == 0x58534350; }
```

Source : https://github.com/pcsx-redux/nugget/blob/main/common/hardware/pcsxhw.h#L31-L37

The API needs DEV8/EXP2 (1f802000 to 1f80207f), which holds the hardware register for the bios POST status, to be expanded to 1f8020ff.
Thus the need to use a custom `crt0.s` if you plan on running your code on real hardware.
The default file provided with the Nugget+PsyQ development environment does that:

```
_start:
    lw    $t2, SBUS_DEV8_CTRL
    lui   $t0, 8
    lui   $t1, 1
_check_dev8:
    bge   $t2, $t0, _store_dev8
    nop
    b     _check_dev8
    add   $t2, $t1
_store_dev8:
    sw    $t2, SBUS_DEV8_CTRL
```

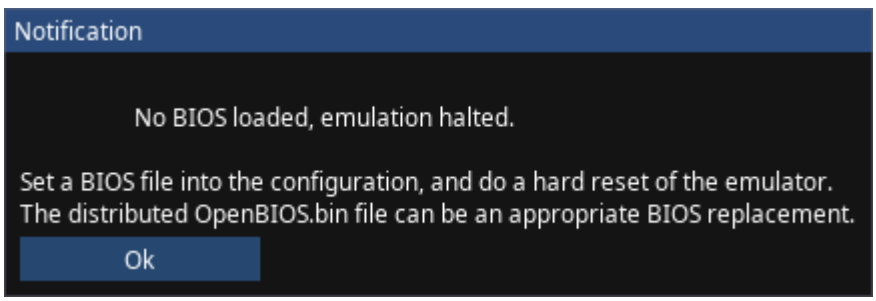
Source : https://github.com/pcsx-redux/nugget/blob/main/common/crt0/crt0.s#L36-L46

6.2 Functions

The following functions are available :

Function	Usage
`pcsx_putc(int c)`	Print ASCII character with code `c` to console/stdout.
`pcsx_debugbreak()`	Break execution (Pause emulation).
`pcsx_execSlot(uint8_t slot)`	Executes Lua function at `PCSX.execSlots[slot]`. The `slot` value can be between 1 and 255, as a `slot` value of 0 is `pcsx_debugbreak()`. If no Lua function exists within a slot, nothing happens.
`pcsx_exit(int code)`	When the emulator was started with `-testmode`, exit the emulator and forward `code` as exit code. Otherwise, log the exit code and pause emulation.
`pcsx_message(const char* msg)`	Create a UI dialog displaying `msg`
`pcsx_checkKernel(int enable)`	Enable or disable kernel checking.
`pcsx_isCheckingKernel()`	Returns truthy if kernel checking is enabled.
`pcsx_present()`	Returns 1 if code is running in PCSX-Redux
`pcsx_initMsan()`	Initialize memory sanitizer system.
`pcsx_resetMsan()`	Reset memory sanitizer system.
`pcsx_msanAlloc(uint32_t size)`	Allocate memory with memory sanitizer.
`pcsx_msanFree(void* ptr)`	Free memory with memory sanitizer.
`pcsx_msanRealloc(void* ptr, uint32_t size)`	Reallocate memory with memory sanitizer.
`pcsx_registerHeapMetadata(const void* metadata)`	Register the address of the psyqo heap metadata, which is used by the PSYQo heap viewer.

Example of a UI dialog created with `pcsx_message()` :



6.2.1 Kernel Checker

The kernel checking feature is used to try and catch unwanted accesses to the kernel, which are usually a sign of a bug in the code, such as a buffer overflow or a null pointer dereference. If the kernel checking feature is enabled, the emulator will break execution and display a message in the console if it detects an unwanted access to the kernel. The following actions are considered unwanted accesses to the kernel:

- Reading or writing to a kernel address from a user-mode address and while not in a kernel-mode context such as while in the ISR. The ISR sets up a stack frame within the kernel space, so callbacks from the kernel and into the user space will be using kernel space as the stack. This means that a null pointer dereference in a callback from the kernel during an interrupt or exception will not be caught by the kernel checking feature.
- An indirect jump to a kernel address from a user-mode address and that isn't 0x40, 0x80, 0xa0, 0xb0, or 0xc0, and that isn't a `jr $ra` instruction. The 0x40 and 0x80 addresses are the break and exception handlers, and the 0xa0, 0xb0, and 0xc0 addresses are the syscall gates. Direct jumps and branches to kernel addresses should be compiler-level problems, so they are not checked for. The `jr $ra` exception to the rule is because callbacks from the kernel will use `jr $ra` to return to the kernel. Optimizations which bypass the `jr $ra` instruction by using a different register to return to the kernel during a callback will cause false positives.

The feature is disabled by default as many games and software will access the kernel in various ways, and it can be enabled by calling `pcsx_checkKernel(1)`. The feature can be disabled by calling `pcsx_checkKernel(0)`, and it is also disabled again on every reset. Since many startup sequences will access the kernel to patch it or clean it, it is recommended to enable the feature after the startup sequence has completed. Some libraries may also access the kernel during their normal operations. The user can simply disable the checker temporarily by toggling it before and after calling such APIs. The kernel space is considered to be all the memory addresses between 0x80000000 and 0x8000ffff. The BIOS is considered to be part of the kernel space in terms of code, so any access to the RAM Kernel space from the BIOS memory space will not trigger any of the kernel checks. The kernel checking feature is only available in the interpreter with the debugger enabled, and it is not available in the dynarec. Trying to enable the feature while using the dynarec, or while the debugger is disabled, will not have any effect.

6.2.2 Memory Sanitizer

The memory sanitizer system of PCSX is inspired of various similar tools. It can detect use-after-frees, buffer overflows, and reads from uninitialized memory. Enabling the memory sanitizer is done through the `pcsx_initMsan()` function call. The emulator will immediately allocate an extra 2GB of memory to store the memory sanitizer data and metadata. Once enabled, the user can call `pcsx_msanAlloc()`, `pcsx_msanFree()`, and `pcsx_msanRealloc()` to allocate, free, and reallocate memory, working as expected from a normal C library. The memory sanitizer will keep track of the memory allocated and will check for the following issues:

- Use-after-frees: If the user tries to access memory that has been freed, the memory sanitizer will break execution and display a message in the console.
- Double frees: If the user tries to free memory that has already been freed, the memory sanitizer will break execution and display a message in the console.
- Corrupted pointer: If the user tries to free or reallocate a pointer that is not a valid pointer, the memory sanitizer will break execution and display a message in the console.
- Buffer overflows: If the user writes to memory before or after the allocated size, up to 1kB, the memory sanitizer will break execution and display a message in the console.
- Reads from uninitialized memory: If the user tries to read from memory that has not been written to first, the memory sanitizer will break execution and display a message in the console.

Internally, the memory sanitizer will allocate memory to the range 0x20000000-0x80000000, which is 1.5GB large. Note that for the use-after-free detection to work, the memory sanitizer will never actually free anything, and so it is possible to run out of memory if the user allocates too much memory. Calling `pcsx_resetMsan()` will re-initialize the memory sanitizer back to its original state. The memory sanitizer is available at all times, and is not affected by the debugger setting nor the dynarec.

7. Web server

A web server can be activated. This allows the use of a REST api to access various features. The server only handles up to HTTP/1.1, without SSL support.

7.1 Activation

You can activate the web server by going to `Configuration > Emulation > Enable Web Server`, or by using the `-webserver` command line flag.

7.2 REST API

By default, the server listens for incoming connection on port 8080, on all network interfaces, so it is reachable at `localhost:8080`, but also from other machines on the network. The port can be changed in the same settings above, or with the `-webserver-port` command line flag.

These GET methods are available:

URL	Function
/api/v1/gpu/vram/raw	Dump VRAM
/api/v1/cpu/ram/raw	Dump RAM
/api/v1/execution-flow	Emulation Status, as a JSON object with the `running`, `isDynarec`, `8mb` and `debugger` fields
/api/v1/cd/info	Information on the loaded disc image, as a JSON object
/api/v1/cd/files?filename=	Dump a file from the loaded disc image
/api/v1/screen/still	Take a screenshot, returned as a PNG image
/api/v1/screen/save?filepath=	Take a screenshot, and save it as a PNG file at the given path. A relative path is relative to the directory holding the settings.
/api/v1/state/usage	List the used save state slots, and the named save states, as a JSON object
/api/v1/state/load?slot= or /api/v1/state/load?name=	Load a save state, either from a slot between 0 and 9, or by name
/api/v1/state/save?slot= or /api/v1/state/save?name=	Save a save state, either to a slot between 0 and 9, or by name
/api/v1/state/delete?slot= or /api/v1/state/delete?name=	Delete a save state, either from a slot between 0 and 9, or by name

The save state endpoints are unavailable when running without the graphical user interface.

The following POST methods are available:

`/api/v1/gpu/vram/raw?x=<value>&y=<value>&width=<value>&height=<value>`

The above needs to also send a form with binary contents. This will partially update the VRAM with the corresponding pixels. The updated rectangle has to be within the 1024x512 16bpp VRAM. The pixels need to be in 16bpp format, meaning the server is expecting exactly `width * height * 2` bytes in the form data. The server will properly parse requests with `Content-Type: multipart/form-data`, but raw bytes in the request body without this header is also acceptable. Any invalid query will result in a 400 error.

`/api/v1/cpu/ram/raw?offset=<value>&size=<value>`

The above needs to also send a form with binary contents, which will update the RAM at the specified offset. Offset is expected to be a number from [0, 0x1FFFFF] in case of running redux with 2MB RAM, or [0, 0x7FFFFF] in case the 8MB memory expansion is enabled. The value of size + offset must not exceed the total space in the RAM.

`/api/v1/assembly/symbols?function=<value>`

Value	Function
reset	Resets the symbols loaded in redux
upload	Uploads a `.map` file to redux

The above expects a `.map` file with symbols and addresses, which will be merged with the current symbols already loaded in redux. The map file should contain a pair of `address symbol` for each line, with the address in hexadecimal. e.g `80010000 Foo` would load the symbol `Foo` in the address `0x80010000`.

`/api/v1/cpu/cache?function=<value>`

Value	Function
flush	Flushes the CPU cache

`/api/v1/execution-flow?function=<value>&type=<value>`

Value	Type	Function
pause	-	Pauses the emulator.
start	-	Starts/Resumes the emulator.
resume	-	Starts/Resumes the emulator.
reset	hard	Hard resets the emulator. Equivalent to a power cycle of the console.
reset	soft	Soft resets the emulator. Equivalent to pressing the reset button.

`/api/v1/cd/patch?filename=<value>`

The above needs to also send a form with binary contents, which will patch the currently loaded iso file with the contents of the form. The server will look for the given filename in the iso file, and patch its contents. All changes are cumulative. If the file is not found, a 404 error will be returned. The file name is case sensitive, and must be a valid ISO9660 filename, which means it can only contain uppercase letters, numbers, and underscores, and ends with `;1`.

For example:

```
$ curl -F file=@newsystem.cnf http://localhost:8080/api/v1/cd/patch?
filename=SYSTEM.CNF;1
```

`/api/v1/cd/patch?sector=<value>&mode=<value>`

The above needs to also send a form with binary contents, which will patch the currently loaded iso file with the contents of the form. The iso sectors starting at the given value will be written to. The `mode` argument is optional, and can be of the following values:

Value	Function
GUESS	Tries to guess the sector's mode. This is the default.
RAW	Writes the full sectors with no decoration, 2352 bytes per sector.
M2_RAW	Writes 2336 bytes per sector, with the first 16 bytes being the subheader.
M2_FORM1	Writes 2048 bytes per sector. Will not update the subheader.
M2_FORM2	Writes 2324 bytes per sector. Will not update the subheader.

All changes are cumulative.

`/api/v1/cd/ppf?function=<value>`

Value	Function
save	Saves the current state of the disc image patches to a PPF file.
clear	Clears the current list of patches.

Additional endpoints can be written in Lua, and are served under `/api/v1/lua/`. See the Lua web server page.

8. SIO1 networking

SIO1 is the PlayStation's second serial interface, connected to the serial port. PCSX-Redux can relay that port over TCP, either by listening for a connection (the SIO1 server) or by connecting to another machine (the SIO1 client). This can be used to link two instances of PCSX-Redux, or to let a program running in the emulator talk to a tool on the host.

8.1 Settings

All the settings are in `Configuration > Emulation`, in the `Emulation Configuration` window:

Setting	Default	Function
Enable SIO1 Server	off	Listens for an incoming TCP connection, and relays it to SIO1.
SIO1 Server Port	6699	Port the server listens on.
Enable SIO1 Client	off	Connects to a remote SIO1 server, and relays the connection to SIO1.
SIO1 Client Host		Address of the server to connect to.
SIO1 Client Port	6699	Port of the server to connect to.
SIO1Mode	Protobuf	How the bytes are carried over TCP, see below.

The server listens on all network interfaces, so it is reachable from other machines on the network, and not only from `localhost`. The client host has to be an IPv4 address, such as `127.0.0.1`; host names are not resolved.

The mode is read when the server or the client starts. Changing it while one of them is running has no effect until it is disabled and enabled again.

The `Reset SIO` button resets the SIO1 registers. The `Reconnect` button is only available while the client is enabled and its connection has dropped. If the client fails to connect in the first place, disable and enable it again instead.

The state of the SIO1 registers can be inspected with `Debug > Misc hardware > Show SIO1 debug`.

8.2 Modes

8.2.1 Protobuf

This is the default mode, meant for linking two instances of PCSX-Redux together, one running the server and the other one running the client. Both sides need to be set to this mode.

Each message is sent as a single byte holding the length of the message, followed by the message itself, encoded as a protobuf `SIOPayload` message:

```
message DataTransfer {
    bytes data = 1;
}

message FlowControl {
    bool dxr = 1;
    bool xts = 2;
}

message SIOPayload {
    DataTransfer data_transfer = 1;
    FlowControl flow_control = 2;
}
```

Every byte the program transmits is sent in its own `data_transfer` message. Whenever the program changes the DTR or RTS bits of the SIO1 control register, a `flow_control` message is sent, with `dxr` holding DTR and `xts` holding RTS. On the receiving side, `dxr` sets the DSR bit of the status register, and `xts` sets the CTS bit.

Received data bytes are only stored in the receive FIFO while the receiving program has RTS set in its control register; otherwise they are dropped. If a message cannot be decoded, the connection is closed, and the server or client needs to be restarted.

8.2.2 Raw

In this mode, the TCP connection carries the serial bytes as they are, with no framing and no flow control: every byte the program writes to the data register is sent as is, and every byte received is placed in the receive FIFO. The CTS and DSR bits are not driven by the remote side. This is the mode to use to talk to a tool on the host, such as a terminal or a script.

Raw mode is only supported by the server. The client does not support it, so select Protobuf before enabling the client.

8.3 Example

The following program configures SIO1 for 8 data bits, no parity, 1 stop bit, then sends back every byte it receives. It polls the status register, with SIO1 interrupts left disabled.

```
#include <stdint.h>

#define SIO1_DATA (*(volatile uint8_t *)0x1f801050)
#define SIO1_STAT (*(volatile uint16_t *)0x1f801054)
#define SIO1_MODE (*(volatile uint16_t *)0x1f801058)
#define SIO1_CTRL (*(volatile uint16_t *)0x1f80105a)
#define SIO1_BAUD (*(volatile uint16_t *)0x1f80105e)

int main(void) {
    SIO1_CTRL = 0x0040;  // reset
    SIO1_MODE = 0x004e;  // reload factor x16, 8 data bits, no parity, 1 stop bit
    SIO1_BAUD = 18;      // 33868800 / (18 * 16) = 117600 bps, close to 115200
    SIO1_CTRL = 0x0027;  // TX enable, DTR, RX enable, RTS

    for (;;) {
        while (!(SIO1_STAT & 0x0002));  // wait until a byte is received
        uint8_t c = SIO1_DATA;
        while (!(SIO1_STAT & 0x0001));  // wait until the transmitter is ready
        SIO1_DATA = c;
    }
}
```

The data register is accessed with 8-bit reads and writes, and the mode, control and baud rate registers with 16-bit writes. See the serial interfaces page of psx-spx for the meaning of each bit.

To try it, set `SIO1Mode` to `Raw`, then enable the SIO1 server, and run the program. Then connect to the server from the host, for instance with netcat:

```
$ nc localhost 6699
```

Anything typed in netcat is sent back by the program.

The emulator relays the bytes regardless of the configured baud rate, which only affects the timing of the SIO1 interrupts.

9. Lua

9.1 Introduction

PCSX-Redux features a Lua API that is available through either a direct Lua console, or a Lua editor, both available through the Debug menu. The Lua VM runs on the main thread, the same one as the UI and the emulated MIPS CPU. As a result, care must be taken to not stall for too long, or the UI will become unresponsive. Using coroutines to handle long-running tasks is recommended, yielding periodically to let the UI perform some work too. The UI is probably going to run at 60FPS or so, which gives a ballpark of 15ms per frame.

9.1.1 Lua engine

The Lua engine that's being used is LuaJIT 2.1 compiled in Lua 5.2 compatibility mode. The Lua 5.1 user manual and LuaJIT user manual are recommended reads. In particular, the bindings heavily make use of LuaJIT's FFI capabilities, which allows for direct memory access within the emulator's process. This means there is little protection against dramatic crashes the LuaJIT FFI engine can cause into the emulator's process, and the user must pay extra attention while manipulating FFI objects. Despite that, the code tries as much as possible to sandbox what the Lua code does, and will prevent crashes on any recoverable exception, including OpenGL and ImGui exceptions.

9.1.2 Lua console

All of the messages coming from Lua should display into the Lua console directly. The input text there is a single line execution, so the user can type one-liner Lua statements and get an immediate result.

9.1.3 Lua editor

The editor allows for more complex, multi-line statements to be written, such as complete functions. The editor will by default auto save its contents on the disc under the filename `pcsx.lua`, which can potentially be a problem if the last statement typed crashed the emulator, as it'll be reloaded on the next startup. It might become necessary to either edit the file externally, or simply delete it to recover from this state.

The auto-execution of the editor permits for rapid development loop, with immediate feedback of what's done.

For complex projects however, it is recommended to split your work into sub-modules, and use the `loadfile` function to load them in your main code. This implies working on your project using an external editor.

9.2 Loaded libraries

9.2.1 Basic Lua

The LuaJIT extensions are fully loaded, and can be used globally. The standard Lua libraries are loaded, and are usable. The `require` function exists, but isn't recommended as the loading of external DLLs might be difficult to properly accomplish. Loading pure Lua files is fine. The `ffi` table is loaded globally, there is no need to `require` it, but it'll work nonetheless. As a side-effect of Luv, Lua-compat-5.3 is loaded.

9.2.2 Dear ImGui

A good portion of ImGui is bound to the Lua environment, and it's possible for the Lua code to emit arbitrary widgets through ImGui. It is advised to consult the user manual of ImGui in order to properly understand how to make use of it. The list of current bindings can be found within the source code. Some usage examples will be provided within the case studies. Additional features and interaction is documented in the rendering page.

9.2.3 OpenGL

OpenGL is bound directly to the Lua API through FFI bindings, loosely inspired and adapted from LuaJIT-OpenCL. Some usage examples can be seen in the CRT-Lottes shader configuration page.

9.2.4 ThorVG

The ThorVG library is bound to the Lua API as the global `tvg` table, through FFI bindings of its C API, with some additional glue code. More explanation can be found in the rendering page.

9.2.5 Luv

For network access and interaction, PCSX-Redux uses libuv internally, and is exposed to the Lua API through Luv, tho its loop is tied to the main thread one, meaning it'll run

only once per frame. There is another layer of network API available through the File API, which is more convenient and faster for simple tasks.

9.2.6 Zlib

The Zlib C-API is exposed through FFI bindings. There is another layer of Zlib API available through the File API, which is more convenient and faster for simple tasks.

9.2.7 FFI-Reflect

The FFI-Reflect library is loaded globally as the `reflect` symbol. It's able to generate reflection objects for the LuaJIT FFI module.

9.2.8 PPrint

The PPrint library is loaded globally as the `pprint` symbol. It's a more powerful `print` function than the one provided by Lua, and can be used to print tables in a more readable way.

9.2.9 Lua-Protobuf

The Lua-Protobuf library is available, but not loaded by default. All of its documented API should be usable straight with no additional work. It has been slightly modified, but nothing that should be visible to the user. There is some limited glue between its API and PCSX's.

9.2.10 luafilesystem

The luafilesystem library is loaded globally as the `lfs` symbol. It's a library that provides access to the filesystem.

9.2.11 LPeg

The LPeg library is available, but not loaded by default. It's a library that provides a pattern-matching library for Lua, which can be useful to create ad-hoc arbitrary parsers.

9.3 Redux basic API

9.3.1 Settings

All of the settings are exposed to Lua via the `PCSX.settings` table. It contains pseudo-tables that are reflections of the internal objects, and can be used to read and write the settings. The exact list of settings can vary quickly over time, so making a full list here would be fruitless. It is possible however to traverse the settings using `pprint` for example. The semantic of the settings is the same as from within the GUI, with the same caveats. For example, disabling the dynamic recompiler requires a reboot of the emulator.

9.3.2 ImGui interaction

PCSX-Redux will periodically try to call the Lua function `DrawImguiFrame` to allow the Lua code to draw some widgets on screen. The function will be called exactly once per actual UI frame draw, which, when the emulator is running, will correspond to the emulated GPU's vsync. If the function throws an exception however, it will be disabled until recompiled with new code.

9.3.3 Events Engine interaction & Execution Contexts

LuaJIT C callbacks aren't called from a safe execution context that can allow for coroutine resuming, and luv's execution context doesn't have any error handling.

It is possible to defer executing code to the main loop of PCSX-Redux, which can (a) resume coroutines and (b) execute code in a safe context. The function

`PCSX.nextTick(func)` will execute the given function in the next main loop iteration. Here's some examples of how to use it:

```
    local captures = {}
    captures.current = coroutine.running()
    captures.callback = function()
        PCSX.nextTick(function()
            captures.callback:free()
            coroutine.resume(captures.current)
        end)
    end
    captures.callback = ffi.cast('void (*)()', captures.callback)
    -- use the C callback somewhere...
```

```
function createClient(ip, port)
  client = luv.new_tcp()

  luv.tcp_connect(client, ip, port, function (err)
    PCSX.nextTick(function()
      assert(not err, err)

      luv.read_start(client, function (err, chunk)
        PCSX.nextTick(function()
          pprint("received at client", {err=err, chunk=chunk})
          assert(not err, err)
          if chunk then
            -- do something with the client
          else
            luv.close(client)
          end
        end)
      end)

      pprint("writing from client")
      luv.write(client, "Hello")
      luv.write(client, "World")

    end)
  end)
  return client
end
```

Of course, this can also delay processing significantly, as the main loop is usually bound to the speed of the UI, which can mean up to 20ms of delay.

9.3.4 Constants

The table `PCSX.CONSTS` contains numerical constants used throughout the rest of the API. Keeping an up to date list here is too exhausting, and it's simpler to print them using `pprint(PCSX.CONSTS)`.

9.3.5 Pads

You can access the pads API through `PCSX.SIO0.slots[s].pads[p]` where `s` is the slot number and `p` is the pad number, both indexed from 1, Lua-style. So `PCSX.SIO0.slots[1].pads[1]` accesses the first pad, and `PCSX.SIO0.slots[2].pads[1]` accesses the second pad.

Each Pad table has the following functions:

```
getButton(button)      -- Returns true if the specified button is pressed.
setOverride(button)    -- Overrides the specified button.
clearOverride(button) -- Clears the override for the specified button.
setAnalogMode(bool)    -- Sets or clears the analog mode of this pad.
map()                  -- Forces the pad to be remapped. Useful after changing pad
settings.
```

The button constants can be found in `PCSX.CONSTS.PAD.BUTTON`.

You can for instance press the button Down on the first pad using the following code:

```
PCSX.SIO0.slots[1].pads[1].setOverride(PCSX.CONSTS.PAD.BUTTON.DOWN)
```

9.3.6 Execution flow

The Lua code has the following API functions available to it in order to control the execution flow of the emulator:

- `PCSX.pauseEmulator()`
- `PCSX.resumeEmulator()`
- `PCSX.softResetEmulator()`
- `PCSX.hardResetEmulator()`

It's also possible to manipulate savestates using the following functions:

- `PCSX.createSaveState()    -- returns a slice representing the savestate`
- `PCSX.loadSaveState(slice)`
- `PCSX.loadSaveState(file)`

When called while the emulated CPU is running, for instance from a `GPU::Vsync` listener or a breakpoint callback, `PCSX.loadSaveState` doesn't load the state right away. The current CPU step finishes against the old state, and the load happens as soon as the CPU loop returns, before any further instruction is executed. Listen to the `ExecutionFlow::SaveStateLoaded` event to know when it happened. When the emulator is paused, the load is immediate.

Additionally, the following function returns a string containing the .proto file used to serialize the savestate:

- `PCSX.getSaveStateProtoSchema()`

Note that the actual savestates made from the UI are gzip-compressed, but the functions above don't compress or decompress the data, so if trying to reload a savestate made from the UI, it'll need to be decompressed first, possibly through the zReader File object.

Overall, this means the following is possible:

```lua
local compiler = require('protoc').new()
local pb = require('pb')

local state = PCSX.createSaveState()
compiler:load(PCSX.getSaveStateProtoSchema())

local decodedState = pb.decode('SaveState', Support.sliceToPBSlice(state))
print(string.format('%08x', decodedState.registers.pc))
```

9.3.7 Messages

The globals `print` and `printError` are available, and will display logs in the Lua Console. You can also use `PCSX.log` to display a line in the general Log window. All three functions should behave the way you'd expect from the normal `print` function in mainstream Lua.

9.3.8 GUI

You can move the cursor within the assembly window and the first memory view using the following functions:

- `PCSX.GUI.jumpToPC(pc)`
- `PCSX.GUI.jumpToMemory(address[, width])`

9.3.9 GPU

You can take a screenshot of the current view of the emulated display using the following:

- `PCSX.GPU.takeScreenShot()`

This will return a struct that has the following fields:

```
struct ScreenShot {
    Slice data;
    uint16_t width, height;
    enum { BPP_16, BPP_24 } bpp;
};
```

The `Slice` will contain the raw bytes of the screenshot data. It's meant to be written out using the `:writeMoveSlice()` method on a `File` object. The `width` and `height` will be the width and height of the screenshot, in pixels. The `bpp` will be either `BPP_16` or `BPP_24`, depending on the color depth of the screenshot. The size of the `data` Slice will be `height * width` multiplied by the number of bytes per pixel, depending on the `bpp`.

You can also get the framerate of the emulated game using the following:

- `PCSX.GPU.getGuestFPS()`

This will return a number, which is how many times the game moved the display start area per second of emulated vsyncs. As games flip their buffers by moving the display start area, this is the framerate of the game itself, as opposed to the framerate of the UI. The value is updated once every 50 or 60 emulated vsyncs, depending on the video mode. Games which keep displaying the same buffer, such as interlaced or single buffered games, will read 0.

9.3.10 SPU

You can play audio from memory through the emulator's audio output using the following:

- `PCSX.SPU.playAudio(source, descriptor)`

The `source` can be a string, a LuaBuffer, a `File` object, or a `Slice`. A `File` is read entirely, without moving its read cursor. The data is copied or decoded before the function returns. The playback is independent of the emulated SPU, and happens whether the emulation is running or paused. The Mute setting also applies to it.

The `descriptor` is a table which describes the format of the data. Its `format` field is mandatory, and can be one of `'pcm'`, `'spu'`, or `'xa'`. All formats accept an optional

`gain` field, which is a non-negative number defaulting to 1.0. Any other field that isn't listed below for the given format is an error.

- `'pcm'`: raw interleaved little endian samples. The data size needs to be a multiple of the frame size.
- `bits`: mandatory, 8 or 16.
- `channels`: mandatory, 1 or 2.
- `rate`: mandatory, the sample rate in Hz, between 1 and 384000.
- `signed`: optional boolean, defaults to true for 16 bits, and false for 8 bits. 16 bits samples have to be signed.
- `'spu'`: a sequence of 16-byte SPU-ADPCM blocks, mono. The data size needs to be a multiple of 16 bytes. Exactly one of these two fields is mandatory:
- `rate`: the sample rate in Hz, between 1 and 176400.
- `pitch`: the SPU pitch value, between 1 and 0x4000, where 0x1000 is 44100Hz.

  The loop flags of the blocks are followed: the playback stops at a block with the loop end flag without the repeat flag, and jumps back to the last loop start block at a block with both the loop end and repeat flags. A single silent block looping onto itself ends the playback, as it is the usual terminator of a one-shot sample. - `'xa'`: XA-ADPCM data. - `sectors`: optional boolean, defaults to false. - When `sectors` is false, the data is a sequence of 128-byte sound groups, and these fields are mandatory: - `bits`: 4 or 8. - `channels`: 1 or 2. - `rate`: 37800 or 18900. - When `sectors` is true, the data is a sequence of 2352-byte raw sectors starting with the CD sync pattern, or of 2336-byte sectors starting at the subheader. Only the audio sectors are played, and the format is read from their subheaders, so `bits`, `channels`, and `rate` are an error. These fields are optional, and only valid in this mode: - `xaFile`: only play the sectors with this file number, between 0 and 255. - `xaChannel`: only play the sectors with this channel number, between 0 and 255.

```
Without these fields, only the file and channel of the first audio sector are
played, since XA files usually interleave several streams. All the played sectors
need to have the same format.
```

The function will throw an error if the descriptor or the data are invalid, or if no audio device is available. Otherwise, it will return a sound object, which has the following methods:

- `:stop()` stops the playback.
- `:isPlaying()` returns true until the playback has finished or has been stopped.
- `:setGain(gain)` changes the gain, which is a non-negative number.

The sound object owns the playback: the sound stops when the object is garbage collected, so it needs to be kept around for as long as the sound should play.

```
-- One second of a 440Hz sine wave, as 16 bits mono samples.
local samples = ffi.new('int16_t[44100]')
for i = 0, 44099 do samples[i] = 8000 * math.sin(2 * math.pi * 440 * i / 44100) end
beep = PCSX.SPU.playAudio(ffi.string(samples, 44100 * 2), { format = 'pcm', bits =
16, channels = 1, rate = 44100 })

-- Playing an XA file, keeping its first stream only.
local file = Support.File.open('MUSIC.XA')
music = PCSX.SPU.playAudio(file, { format = 'xa', sectors = true, gain = 0.5 })
file:close()
```

9.3.11 Loading and executing code

While the basic Lua functions `dofile` and `loadfile` exist, some alternative functions are available to load and execute code in a more flexible way.

- `Support.extra.addArchive(filename)` will load the given zip file, and will make it available to the `Support.extra.dofile` function. It is equivalent to the `-archive` command line flag. Note that if a file named `autoexec.lua` is found in the zip file, it will be executed automatically.
- `Support.extra.dofile(filename)` will load the given file, and execute it. It is equivalent to `dofile`, but will also search for the file next to the currently loaded Lua file which is calling this function, and will also search for the file in all of the loaded zip files, either through the command line, or through the `Support.extra.addArchive` function.
- `Support.extra.loadfile(filename)` will load the given file, and return a function that can be called to execute the file. It is equivalent to `loadfile`, but has the same file search algorithm as `Support.extra.dofile`.
- `Support.extra.open(filename)` will open the given file as read only, and return a `File` object. It is roughly equivalent to `Support.File.open`, but has the same file search algorithm as `Support.extra.dofile`.

If given the following directory structure:

```
.
└── bar.zip
    ├── test/baz.lua
    └── test2/qux.lua
```

If `test/baz.lua` contains the following code:

```
Support.extra.dofile('../test2/qux.lua')
```

Then running the following code:

```
Support.extra.addArchive('bar.zip')
Support.extra.dofile('test/baz.lua')
```

Will first load `test/baz.lua` from the zip file `bar.zip`, run it, which will in turn load `test2/qux.lua` from the zip file `bar.zip` again, and execute it.

This allows distributing complex "mods" as zip files, which can be loaded and executed from the command line or the console.

9.3.12 Miscellaneous

- `PCSX.quit([code])` schedules the emulator to quit. It's not instantaneous, and will only quit after the current block of Lua code has finished executing, which will be before the next main loop iteration. The `code` parameter is optional, and will be the exit code of the emulator. If not specified, it'll default to 0.
- `PCSX.getCPUCycles()` returns an unsigned 64-bit number indicating how many CPU cycles have elapsed. This can be paired with the `PCSX.CONSTS.CPU.CLOCKSPEED` constant to determine how much emulated time has passed.
- `PCSX.Adpcm.NewEncoder` will return an Adpcm encoder object. The object has the following methods:
- `:reset([mode])` will reset the encoder, and set the mode to the given mode. The mode can be `'Normal'`, `'XA'`, `'High'`, `'Low'`, `'FourBits'`. The default mode is `'Normal'`, which enables all the filters available in the SPU. The `'XA'` mode limits the encoder to the filters available in the XA ADPCM format. The `'High'` mode uses the high-pass filter, and the `'Low'` mode uses the low-pass filter. The `'FourBits'` mode forces plain 4-bit Adpcm encoding.
- `:processBlock(inData, [outData], [channels])` will encode the given ffi input buffer, and write the result to the given ffi output buffer. The input buffer should be a buffer of 16-bit signed integers, and the output buffer should be a buffer of 16-bit signed integers. The channels parameter is optional, and will default to 1. The input buffer should contain exactly 28 samples per channel, interleaved when stereo, and so does the output buffer. If the output buffer is not given, the function will return a new buffer with the result. LuaBuffers are also accepted as input and output buffers. The function will return three values: the output buffer, the filter index used, and the shifting used. With 2 channels, it will return five values: the output buffer, then the filter index and shifting used for the first channel, then the filter index and shifting used for the second channel. The function is intended to be used as an intermediate

computation step, and the output still needs to be processed into 4 bits or 8 bits samples.

- `:processSPUBlock(inData, [outData], [blockAttribute])` will encode the given ffi input buffer, and write the result to the given ffi output buffer. The input buffer should be a buffer of 16-bit signed integers, and the output buffer should be a buffer which is at least 16 bytes large. The blockAttribute parameter is optional, and will default to `'OneShot'`. The input buffer should contain exactly 28 samples. If the output buffer is not given, the function will return a new buffer with the result. LuaBuffers are also accepted as input and output buffers. The function will return the encoded block, suitable for SPU usage. The `blockAttribute` parameter can be one of the following strings: `'OneShot'`, `'OneShotEnd'`, `'LoopStart'`, `'LoopBody'`, `'LoopEnd'`.
- `:finishSPU([outData])` will write the opinionated end of sample looping block, as prescribed by the original Sony API. The output buffer should be a buffer which is at least 16 bytes large. If the output buffer is not given, the function will return a new buffer with the result. LuaBuffers are also accepted as output buffers. The function will return the encoded block, suitable for SPU usage.
- `:processXABlock(inData, [outData], [xaMode], [channels])` will encode the given ffi input buffer, and write the result to the given ffi output buffer. The input buffer should be a buffer of 16-bit signed integers, and the output buffer should be a buffer which is at least 128 bytes large. Note that a MODE2 FORM2 XA sector requires subheaders and 18 of these blocks. The xaMode parameter is optional, and will default to `'XAFourBits'`. The other valid value is `'XAEightBits'`. It will defines the encoding output between either 4-bit and 8-bit. The channels parameter is optional, and will default to 1. If the output buffer is not given, the function will return a new buffer with the result. LuaBuffers are also accepted as input and output buffers. The function will return the encoded block, suitable for XA usage. The amount of required input samples varies depending of the number of channels and the encoding mode:
- 4-bit mono: 224 samples aka 448 bytes
- 4-bit stereo: 112 samples aka 448 bytes
- 8-bit mono: 112 samples aka 224 bytes
- 8-bit stereo: 56 samples aka 224 bytes
- `PCSX.Adpcm.NewDecoder` will return an Adpcm decoder object. The object has the following methods:

- `:reset()` will reset the decoder history.
- `:decodeSPUBlock(inData, [outData])` will decode a single 16-byte SPU-ADPCM block into 28 16-bit signed samples. The input can be a LuaBuffer, a string, or an ffi pointer, and needs to be at least 16 bytes large. The output can be a LuaBuffer or an ffi pointer, and needs to be at least 56 bytes large. If the output buffer is not given, the function will return a new LuaBuffer with the result. The function will return two values: the output buffer, and the flags byte of the block, which contains the loop flags.
- `:decodeXASoundGroup(inData, [outData], [bitsPerSample], [channels])` will decode a single 128-byte XA-ADPCM sound group into 16-bit signed samples. The input can be a LuaBuffer, a string, or an ffi pointer, and needs to be at least 128 bytes large. The output can be a LuaBuffer or an ffi pointer. If the output buffer is not given, the function will return a new LuaBuffer with the result. The bitsPerSample parameter is optional, can be 4 or 8, and will default to 4. The channels parameter is optional, can be 1 or 2, and will default to 1. Stereo output is interleaved, left first. The function will return two values: the output buffer, and the number of samples written, which is 224 for 4-bit, and 112 for 8-bit, stereo included. An XA sector contains 18 of these sound groups.

Using the encoder to process an input audio file is as simple as:

```
function encodeAudioLoop(inputFile, outputFile)
  local closeInput = false
  if type(inputFile) == 'string' then
    inputFile = Support.File.open(inputFile)
    closeInput = true
  end
  local audio = Support.File.ffmpegAudioFile(inputFile, {
    channels = 'Mono',
    frequency = 22050
  })

  local closeOutput = false
  if type(outputFile) == 'string' then
    outputFile = Support.File.open(outputFile, 'TRUNCATE')
    closeOutput = true
  end
  local blockCount = math.floor(audio:size() / (2 * 28))
  local bufferIn = ffi.new('int16_t[28]')
  local bufferOut = Support.NewLuaBuffer(16)

  local encoder = PCSX.Adpcm.NewEncoder()
  encoder:reset 'Normal'

  for i = 1, blockCount do
    audio:read(bufferIn, 28 * 2)
    local blockType = 'LoopBody'
    if i == 1 then blockType = 'LoopStart' end
    if i == blockCount then blockType = 'LoopEnd' end
    encoder:processSPUBlock(bufferIn, bufferOut, blockType)
    outputFile:write(bufferOut)
  end

  if closeInput then
    inputFile:close()
  end
  if closeOutput then
    outputFile:close()
  end
  audio:close()
end
```

9.4 Rendering

PCSX-Redux is entirely running as an OpenGL3 application. All of its aspects, including the UI elements, are rendered using OpenGL primitives. This means there is very little boundaries between the various rendered elements on the screen.

The rendering of the UI is done through ImGui, and a chunk of its API is bound to Lua using bindings.

A good portion of the OpenGL3 API is also bound to Lua, as well as the ThorVG library.

9.4.1 Emulated GPU rendering pipeline

The content of the Output region is rendered in two steps. The first step is called the "Offscreen rendering", and is done during the emulated GPU vsyncs. Its job is to flush the contents of the VRAM texture to an offscreen texture, which may be of a different resolution. The resolution of the offscreen texture should be pixel perfect with that of the Output region. By default, the associated shader with this operation should only do a simple copy and interpolation, but as the first stage of the rendering pipeline, this can be used for some first pass output effect such as the first pass of a crt shader.

The second step is called the "Output rendering", and is done every time the UI wants to refresh its display, which may or may not be at the same time as the emulated vsync. The resolution of the Output region will match exactly the resolution of the input texture, and the default shader should simply copy all the texels without any sort of interpolation, but as the second stage of the rendering pipeline, this can still be used for the second pass output effect.

The crt-lottes implementation leverages these two passes to do the full CRT-like output.

9.4.2 Shader editor

The shader editor is a simple text editor that allows to edit the shader code. It is not a full IDE, and it is not meant to be. Its point is to do quick iterations on the shader code, and to be able to see the result of the changes in real time.

The shader editor is split in 3 regions:

- The left tab is the vertex shader code. It is technically editable, but there shouldn't be much reason to edit it.
- The middle tab is the fragment shader code. This is the main shader code. It is editable, and the changes will be reflected in real time.
- The right tab is the Lua invoker code. This is the code that will be executed under multiple circumstances. It is editable, and the changes will be reflected in real time.

The Lua invoker code will be compiled and executed in a soft sandbox environment. The code can still access already created globals and mutate them, but any newly created global will be kept within the sandbox and won't be accessible from other Lua code. When the `Draw` function returns true, all these globals will be saved to a JSON file, and they will be restored the next time the shader is compiled.

When the shaders are compiled, the Vertex and Fragment shader code will be compiled together, and if the resulting program is valid, the Lua invoker code will be compiled and executed. If the Lua code fails to compile or execute, the error will be displayed in the shader editor, but the shader program itself will still be used.

This compilation order allows the Lua code to access the shader program uniforms, and to set them up as needed. The global `shaderProgramID` will be available to the Lua code, and will contain the ID of the shader program.

The code is expected to export a few functions:

- `Draw`, which will be called periodically within the ImGui context, allowing to draw UI elements. The global `configureme` will be set to true when the user selects the "Configure Shaders" menu item. This allows to display a configuration UI to the user during this function call.
- `Image(textureID, srcSizeX, srcSizeY, dstSizeX, dstSizeY)`, which will be called periodically within the ImGui context, when the emulator needs to draw the texture `textureID` at the given size. The texture ID is the OpenGL texture ID, and the size is in pixels. The code is at best expected to do a simple call to `imgui.Image(textureID, dstSizeX, dstSizeY, 0, 0, 1, 1)` to draw the texture. For the Emulated GPU Pipeline, this function will only be called on the Output shader, when being drawn to the Output region. As the function will be called during the ImGui context, it can capture certain ImGui state, such as the current ImGui cursor position, and use it to draw additional UI elements. Note that as with any normal ImGui function, this isn't the moment when the UI elements are actually drawn, but rather when the UI elements are queued to be drawn, meaning this isn't when the shader program will be executed, which is the point of the next function.
- `BindAttributes(textureID, shaderProgramID, srcLocX, srcLocY, srcSizeX, srcSizeY, dstSizeX, dstSizeY)` will be called when the shader program is about to be executed, and needs to bind the attributes. The texture ID is the OpenGL texture ID, and the shader program ID is the OpenGL shader program ID. The location and sizes are only passed for the Emulated GPU Pipeline, when the Offscreen shader is being executed, as it needs to grab a portion of the VRAM texture to be rendered to the offscreen texture. The source location and size are normalized texture coordinates within the VRAM texture, while the destination size is in pixels.

Additionally, it is possible to programmatically set the content of the editors using the following methods:

```
PCSX.GUI.OffscreenShader.setDefaults()
PCSX.GUI.OffscreenShader.setTextVS(text)
PCSX.GUI.OffscreenShader.setTextPS(text)
PCSX.GUI.OffscreenShader.setTextL(text)
PCSX.GUI.OutputShader.setDefaults()
PCSX.GUI.OutputShader.setTextVS(text)
PCSX.GUI.OutputShader.setTextPS(text)
PCSX.GUI.OutputShader.setTextL(text)
```

The `setDefaults` method will set the default shader code, and the `setText*` methods will set the shader code to the given string. The `text` argument can be either an actual string, or a `File` object.

9.4.3 ImGui

The ImGui API is bound to Lua, and can be used to draw UI elements. The ImGui API is documented on the ImGui source code. There is also an interactive manual available.

Not all functions are necessarily bound to Lua, and one can check the bindings code to see which functions are bound, and why some functions are not bound.

The main reason for not binding a function is that its arguments or return values are not trivial to bind. For example, the `ImGui::Text` C++ function is not bound, as it takes a variadic number of arguments, which is not possible to bind in Lua easily. Instead, the `ImGui::TextUnformatted` C++ function is bound, which takes a single string argument.

The emulator will periodically try to call the global function `DrawImguiFrame` with no arguments. If the function is not defined, nothing will happen. If the function fails to execute, it will be removed from the global environment, and the emulator will stop trying to call it until a new global is defined.

The `DrawImguiFrame` function is expected to call the `imgui.Begin` function to create a new ImGui window, as there is no default window created by the emulator for the Lua context. The `DrawImguiFrame` function is also expected to call the `imgui.End` function as normal with the ImGui API.

Some extra functions are bound to Lua beyond the API listed above:

- `imgui.extra.ImVec2.New(x, y)` will create a new FFI `ImVec2` object. The `ImVec2` object is a simple struct with two fields, `x` and `y`. The `New` function takes two optional arguments, the `x` and `y` values, and returns the new `ImVec2` object.
- `imgui.extra.getCurrentViewportId()` will return the current viewport ID. Viewports in ImGui are a way to split the ImGui context into multiple independent contexts, and the viewport ID is a unique identifier for each viewport. Basically, each viewport is a physical window from the operating system, and it can contain one or more ImGui windows.
- `imgui.extra.getViewportFlags(id)` will return the viewport flags for the specified viewport. The viewport flags are of the type `ImGuiViewportFlags_` in the ImGui C++ API, and is a bitmask of flags, which are exposed as individual values in the Lua generated bindings.
- `imgui.extra.setViewportFlags(id, flags)` will set the viewport flags for the specified viewport. The proper usage of this function is to call `imgui.extra.getViewportFlags` to get the current flags, modify the flags as needed, and then call `imgui.extra.setViewportFlags` to set the new flags.
- `imgui.extra.getViewportPos(id)` will return the position of the specified viewport. The position is returned as an `ImVec2` object.
- `imgui.extra.getViewportSize(id)` will return the size of the specified viewport. The size is returned as an `ImVec2` object.
- `imgui.extra.getViewportWorkPos(id)` will return the work position of the specified viewport. The work position is returned as an `ImVec2` object.
- `imgui.extra.getViewportWorkSize(id)` will return the work size of the specified viewport. The work size is returned as an `ImVec2` object.
- `imgui.extra.getViewportDpiScale(id)` will return the DPI scale of the specified viewport. The DPI scale is returned as a number. A value of 1.0 means that the DPI scale for this viewport is 100%.
- `imgui.extra.InputText(label, text[, flags])` will create an input text widget. The `label` is the label to display next to the input text, and the `text` is the current text to display in the input text. The `flags` are optional, and are the same flags as the ones used by the `imgui::InputText` C++ function. The function will return a boolean indicating if the text has changed or not, and the new text.

- `imgui.extra.InputTextWithHint(label, hint, text[, flags])` will create an input text widget. The `label` is the label to display next to the input text, and the `hint` is the hint to display in the input text when the text is empty. The `text` is the current text to display in the input text. The `flags` are optional, and are the same flags as the ones used by the `imgui::InputTextWithHint` C++ function. The function will return a boolean indicating if the text has changed or not, and the new text.
- `imgui.extra.logText(text)` will call the `imgui::LogText` C++ function, which will add the given text to current log buffer.
- `PCSX.GUI.useMainFont()` will call the `imgui::PushFont` C++ function with the proportional font. It will need to be followed by a call to `imgui.PopFont()`.
- `PCSX.GUI.useMonoFont()` will call the `imgui::PushFont` C++ function with the monospace font. It will need to be followed by a call to `imgui.PopFont()`.

Safety

The ImGui API will frequently assert and crash the process if the API calls are imbalanced. For example, if the `imgui.BeginTable` function is called without calling the `imgui.EndTable` function, the process will most likely crash.

This can be problematic when using the ImGui API from Lua, as the Lua code is not able to catch the crash, and the process will crash without any indication of what went wrong.

The main reason for imbalanced API calls can be attributed to the user code throwing an exception, which will cause the Lua code to unwind the stack, and the ImGui API will not be able to properly clean up its state.

For example, consider the following code:

```
function DrawImguiFrame()
    if imgui.Begin("My Window") then
        error("Something went wrong")
    end
    imgui.End()
end
```

The `imgui.Begin` function will be called, but the `imgui.End` function will not be called, as the `error` function will unwind the stack, and the `imgui.End` function will never be called.

In order to mitigate this, safe wrappers are provided for all of the ImGui Begin*/End* functions, except `imgui.BeginItemTooltip`. Each wrapper takes the same arguments as the corresponding Begin* function, followed by a function to call for the contents. The safe wrappers will catch any exception thrown by the user code, and will call the corresponding End* function. The error will be rethrown after the End* function is called. The wrapped function will only be called if the Begin* function returned true, and receives the same arguments as the Begin* function. The wrapper returns the values returned by the Begin* function.

The End* function is called following the ImGui rules for each pair:

- `imgui.safe.Begin`, `imgui.safe.BeginChild` and `imgui.safe.BeginChild_4` always call `imgui.End` or `imgui.EndChild`, whatever the Begin* function returned.
- `imgui.safe.BeginDisabled` and `imgui.safe.BeginGroup` always call the wrapped function, since `imgui.BeginDisabled` and `imgui.BeginGroup` return nothing, and then always call `imgui.EndDisabled` or `imgui.EndGroup`.
- All the other wrappers only call the End* function if the Begin* function returned true.

The example above can be rewritten as:

```
function DrawImguiFrame()
    imgui.safe.Begin("My Window", function()
        error("Something went wrong")
    end)
end
```

9.4.4 ThorVG

The ThorVG library is bound to Lua under the global `tvg` table, and can be used to draw arbitrary vector graphics, SVG and Lottie files, PNG and JPEG images, and text on top of the emulator. The rendering happens after the ImGui rendering, meaning that it will be on top of the ImGui rendering, regardless of the order in which the ThorVG and ImGui functions are called.

The binding exposes the ThorVG object model directly, through its C API. Objects are created with the following constructors:

- `tvg.Shape()`
- `tvg.Scene()`
- `tvg.Picture()`
- `tvg.Text()`
- `tvg.LinearGradient([x1, y1, x2, y2])`
- `tvg.RadialGradient([cx, cy, r[, fx, fy[, fr]]])`, where `fx` and `fy` default to `cx` and `cy`, and `fr` defaults to 0.
- `tvg.Animation()`
- `tvg.LottieAnimation()`

Their methods are named after the C functions, in camel case, without the `tvg_<type>_` prefix, and without the object argument. For example, `tvg_shape_append_rect(shape, ...)` becomes `shape:appendRect(...)`, and `tvg_paint_translate(paint, x, y)` becomes `paint:translate(x, y)`. Shapes, scenes, pictures and texts also have all the `tvg_paint_*` methods. Gradients have the `tvg_gradient_*` methods, plus `:setLinear`, `:getLinear`, `:setRadial` and `:getRadial`. Animations have the `tvg_animation_*` and `tvg_lottie_animation_*` methods. The methods return the raw `Tvg_Result` value of the C function, where 0 means success. The raw C functions are also available in `tvg.C`.

A few methods have default arguments on top of the C API:

- `shape:appendRect(x, y, w, h[, rx[, ry[, cw]]])`: `rx` defaults to 0, `ry` defaults to `rx`, and `cw` defaults to true.
- `shape:appendCircle(cx, cy, rx[, ry[, cw]])`: `ry` defaults to `rx`, and `cw` defaults to true.
- `shape:setFillColor(r, g, b[, a])` and `shape:setStrokeColor(r, g, b[, a])`: the components are integers between 0 and 255, and `a` defaults to 255.
- `gradient:setColorStops(stops)` also accepts a table of `{ offset, r, g, b[, a] }` entries, where `a` defaults to 255.
- `shape:setGradient(gradient)`, `shape:setStrokeGradient(gradient)` and `text:setGradient(gradient)` use a copy of the gradient, so the gradient can be reused afterwards.

Additionally, the following functions are available:

- `tvg.getViewportScene([viewportId])` returns the scene rendered on top of the given ImGui viewport, or `nil` if ThorVG isn't available. The viewport defaults to the one of the ImGui window currently being drawn, as returned by `imgui.extra.getCurrentViewportId()`. Paints in this scene are expressed in ImGui coordinates, which are absolute screen coordinates when multi-viewports are enabled.
- `tvg.drawBezierArrow(width, p1, c1, c2, p2[, innerColor[, outerColor]])` draws an arrow on top of the current ImGui viewport, for the current frame only. The points are tables or objects with `x` and `y` fields, such as the ones created by `imgui.extra.ImVec2.New`. The colors are tables with either `r`, `g`, `b`, `a` fields or 4 array entries, with components between 0.0 and 1.0. The inner color defaults to opaque white, and the outer color to opaque grey.
- `tvg.loadFont(path)` and `tvg.unloadFont(path)` load and unload a TTF or OTF font file. A loaded font is referred to by its file name without the extension. The emulator's font is loaded at startup, so `text:setFont('NotoSans-Regular')` works without loading a font first.

The rendering is retained, not immediate: a paint added to the viewport scene with `scene:add(paint)` is displayed on every frame until it is removed with `scene:remove(paint)`, and it can be modified in place in between. Lua holds its own

reference on the objects it creates, and releases it when they are garbage collected, but adding a paint to a scene keeps it alive as long as it is in the scene. Remove the paints from the scene when they should no longer be displayed.

9.4.5 Example of using everything together

As the vector graphics are drawn on top of the ImGui viewports, it is required to use the ImGui API to draw some UI, grab the positions of the vector graphics to add, and then update some ThorVG paints accordingly.

The following example will draw a red rectangle in the middle of the Output region. The rectangle will be 100x100 pixels in size, and will be drawn on top of the emulator rendering. It should follow around the Output region when resizing or moving the window.

In order to work, this example requires the code to be executed in the `Image` function of the Output shader invoker, so we can get the position of the Output region to draw to.

```
local square = tvg.Shape()
square:setFillColor(255, 0, 0)
local scene

function Image(textureID, srcSizeX, srcSizeY, dstSizeX, dstSizeY)
    -- The top left corner of the image, in ImGui coordinates.
    local x, y = imgui.GetCursorScreenPos()

    -- The Output window may move to another viewport, so the square
    -- needs to follow it into the matching scene.
    local current = tvg.getViewportScene()
    if current ~= scene then
        if scene then scene:remove(square) end
        scene = current
        if scene then scene:add(square) end
    end

    -- Resetting a shape clears its path, but keeps its colors.
    square:reset()
    square:appendRect(x + dstSizeX / 2 - 50, y + dstSizeY / 2 - 50, 100, 100)
    imgui.Image(textureID, dstSizeX, dstSizeY, 0, 0, 1, 1)
end
```

9.5 File API

9.5.1 Introduction & Rationale

While the normal Lua io API is loaded, there's a more powerful API that's more tightly integrated with the rest of the PCSX-Redux File handling code. It's an abstraction class that allows seamless manipulation of various objects using a common API.

The File objects have different properties depending on how they are created and their intention. But generally speaking, the following rules apply:

- Files are reference counted. They will be deleted when the reference count reaches zero. The Lua garbage collector will only decrease the reference count.
- Whenever possible, writes are deferred to an asynchronous thread, making writes return basically instantly. This speed up comes at the trade off of data integrity, which means writes aren't guaranteed to be flushed to the disk yet when the function returns. Data will always have integrity internally within PCSX-Redux however, and when exiting normally, all data will be flushed to the disk.
- Some File objects can be cached. When caching, reads and writes will be done transparently, and the cache will be used instead of the actual file. This will make reads return basically instantly too.
- The Read and Write APIs can haul LuaBuffer objects. These are Lua objects that can be used to read and write data to the file. You can construct one using the `Support.NewLuaBuffer(size)` function. They can be cast to strings, and can be used as a table for reading and writing bytes off of it, in a 0-based fashion. The length operator will return the size of the buffer. The methods `:maxsize()` and `:resize(size)` are available. They also have a `.pbSlice` property that implicitly converts them to a Lua-Protobuf's `pb.slice`, which can then be passed to `pb.decode`.
- The Read and Write APIs can also function using Lua-Protobuf's buffers and slices respectively.
- If the file isn't closed when the file object is destroyed, it'll be closed then, but letting the garbage collector do the closing is not recommended. This is because the garbage collector will only run when the memory pressure is high enough, and the file handle will be held for a long time.
- When using streamed functions, unlike POSIX files handles, there's two distinct seeking pointers: one for reading and one for writing.

9.5.2 Common API for all File objects

All File objects have the following API attached to them as methods:

Closes and frees any associated resources. Better to call this manually than letting the garbage collector do it:

```
:close()
```

Reads from the File object and advances the read pointer accordingly. The return value depends on the variant used.

```
:read(size)            -- returns a LuaBuffer
:read(ptr, size)       -- returns the number of bytes read, ptr has to be a cdata of
pointer type
:read(buffer)          -- returns the number of bytes read, and adjusts the buffer's
size
:read(pb_buffer, size) -- returns the number of bytes read, while appending to the
pb_buffer's existing data
:gets()                -- returns a string, up to the next newline character
```

Reads from the File object at the specified position. No pointers are modified. The return value depends on the variant used, just like the non-At variants above.

```
:readAt(size, pos)
:readAt(ptr, size, pos)
:readAt(buffer, pos)
:readAt(pb_buffer, pos)
```

Writes to the File object. The non-At variants will advances the write pointer accordingly. The At variants will not modify the write pointer, and simply write at the requested location. Returns the number of bytes written. The `string` variants will in fact take any object that can be transformed to a string using `tostring()`.

```
:write(string)
:write(buffer)
:write(slice)
:write(pb_slice)
:write(ptr, size)
:writeAt(string, pos)
:writeAt(buffer, pos)
:writeAt(slice, pos)
:writeAt(pb_slice, pos)
:writeAt(ptr, size, pos)
```

Note that in this context, `pb_slice` and `pb_buffer` refer to Lua-Protobuf's `pb.slice` and `pb.buffer` objects respectively.

Some APIs may return a `Slice` object, which is an opaque buffer coming from C++. The `write` and `writeAt` methods can take a `Slice`. It is possible to write a slice to a file in a zero-copy manner, which will be more efficient:

```
:writeMoveSlice(slice)
:writeAtMoveSlice(slice, pos)
```

After which, the slice will be consumed and not reusable. The `Slice` object is convertible to a string using `tostring()`, and also has, among others, the members `data`, which is a `const void*`, and `size`. Once consumed by the `MoveSlice` variants, the size of a slice will go down to zero.

Finally, it is possible to convert a `Slice` object to a `pb.slice` one using the `Support.sliceToPBSlice` function. However, the same caveats as for normal `pb.slice` objects apply: it is fragile, and will be invalidated if the underlying Slice is moved or destroyed, so it is recommended to use it as a temporary object, such as an argument to `pb.decode`. Still, it is a much faster alternative to calling `tostring()` which will make a copy of the underlying slice.

The following methods manipulate the read and write pointers. All of them return their corresponding pointer. The `wheel` argument can be of the values `'SEEK_SET'`, `'SEEK_CUR'`, and `'SEEK_END'`, and will default to `'SEEK_SET'`.

```
:rSeek(pos[, wheel])
:rTell()
:wSeek(pos[, wheel])
:wTell()
```

These will query the corresponding File object.

```
:size()      -- Returns the size in bytes, if possible. If the file is not seekable,
will throw an error.
:seekable()  -- Returns true if the file is seekable.
:writable()  -- Returns true if the file is writable.
:eof()       -- Returns true if the read pointer is at the end of file.
:failed()    -- Returns true if the file failed in some ways. The File object is
defunct if this is true.
:cacheable() -- Returns true if the file is cacheable.
:caching()   -- Returns true if caching is in progress or completed.
:cacheProgress() -- Returns a value between 0 and 1 indicating the progress of the
caching operation.
```

If applicable, this will start caching the corresponding file in memory.

```
:startCaching()
```

Same as above, but will suspend the current coroutine until the caching is done. Cannot be used with the main thread.

```
:startCachingAndWait()
```

Duplicates the File object. This will re-open the file, and possibly duplicate all ressources associated with it.

```
:dup()
```

Creates a read-only view of the file starting at the specified position, spanning the specified length. The view will be a new File object, and will be a view of the same underlying file. The default values of start and length are 0 and -1 respectively, which will effectively create a view of the entire file. The view may have less features than the underlying file, but will always be seekable, and keep its seeking position independent of the underlying file. The view will hold a reference to the underlying file.

```
:subFile([start[, length]])
```

In addition to the above methods, the File API has these helpers, that'll read or write binary values off their corresponding stream position for the non-At variants, or at the indicated position for the At variants. All the values will be read or stored in Little Endian, regardless of the host's endianness.

```
:readU8(), :readU16(), :readU32(), :readU64(),
:readI8(), :readI16(), :readI32(), :readI64(),
:readU8At(pos), :readU16At(pos), :readU32At(pos), :readU64At(pos),
:readI8At(pos), :readI16At(pos), :readI32At(pos), :readI64At(pos),
:writeU8(val), :writeU16(val), :writeU32(val), :writeU64(val),
:writeI8(val), :writeI16(val), :writeI32(val), :writeI64(val),
:writeU8At(val, pos), :writeU16At(val, pos), :writeU32At(val, pos), :writeU64At(val,
pos),
:writeI8At(val, pos), :writeI16At(val, pos), :writeI32At(val, pos), :writeI64At(val,
pos),
```

9.5.3 Creating File objects

The Lua VM can create File objects in different ways:

```
Support.File.open(filename[, type])
Support.File.buffer()
Support.File.buffer(ptr, size[, type])
Support.File.mem4g()
Support.File.uvFifo(address, port)
Support.File.zReader(file[, size[, raw]])
```

Basic files

The `open` function will function on filesystem and network URLs, while the `buffer` function will generate a memory-only File object that's fully readable, writable, and seekable. The `type` argument of the `open` function will determine what happens exactly. It's a string that can have the following values:

- `READ`: Opens the file for reading only. Will fail if the file does not exist. This is the default type.
- `TRUNCATE`: Opens the file for reading and writing. If the file does not exist, it will be created. If it does exist, it will be truncated to 0 size.
- `CREATE`: Opens the file for reading and writing. If the file does not exist, it will be created. If it does exist, it will be left untouched.
- `READWRITE`: Opens the file for reading and writing. Will fail if the file does not exist.
- `DOWNLOAD_URL`: Opens the file for reading only. Will immediately start downloading the file from the network. The `filename` argument will be treated as a URL. The curl is the backend for this feature, and its url schemes are supported. The progress of the download can be monitored with the `:cacheProgress()` method.
- `DOWNLOAD_URL_AND_WAIT`: As above, but suspends the current coroutine until the download is done. Cannot be used with the main thread.

Buffers

When calling `.buffer()` with no argument, this will create an empty read-write buffer. When calling it with a cdata pointer and a size, this will have the following behavior, depending on type:

- `READWRITE` (or no type): The memory passed as an argument will be copied first.
- `READ`: The memory passed as an argument will be referenced, and the lifespan of said memory needs to outlast the File object. The File object will be read-only.
- `ACQUIRE`: It will acquire the pointer passed as an argument, and free it later using `free()`, meaning it needs to have been allocated using `malloc()` in the first place.

The `.mem4g()` constructor will return a sparse buffer that has a virtual 4GB span. It can be used to read and write data in the 4GB range, but will not actually allocate any memory until the data is actually written to. This is useful for doing operations that are similar to that of the PlayStation memory. The `.mem4g()` constructor will return a File object that's fully readable, writable, and seekable. Its size will always be 4GB. The returned object will have 3 additional methods:

- `:lowestAddress()`: Returns the lowest address that has been written to.
- `:highestAddress()`: Returns the highest address that has been written to.
- `:actualSize()`: Returns the size of the buffer, which is the highest address minus the lowest address.

This is a useful object to use with the `:subFile()` method, as it will allow you to create a view of a specific range of the 4GB memory. Specifically, `obj:subFile(obj:lowestAddress(), obj:actualSize())` will create a view of the entire memory that has been written to.

Network streams

The `uvFifo` function will create a File object that will read from and write to the specified TCP address and port after connecting to it. The `:failed()` method will return true in case of a connection failure. The address is a string, and must be a strict IP address, no hostnames allowed. The port is a number between 1 and 65535 inclusive. As the name suggests, this object is a FIFO, meaning that incoming bytes will be consumed by any read operation. The `:size()` method will return the number of bytes in the FIFO. Writes will be immediately sent over. There are no reception guarantees, as

the other side might have disconnected at any point. The `:eof()` method will return true when the opposite end of the stream has been disconnected and there's no more bytes in the FIFO. In addition to the normal `File` API, a `uvFifo` has a method called `:isConnecting()`, which returns a boolean indicating the fifo is still connecting, meaning it's possible to verify if the fifo has successfully connected using the boolean expression `not fifo:isConnecting() and not fifo:failed()`.

Compressed streams

The `zReader` function will create a read-only File object which decompresses the data from the specified File object. The `file` argument is a File object, and the `size` argument is an optional number that will be used to determine the size of the decompressed data. If not specified, the resulting file won't be seekable, and its `:size()` method won't work, but the file will be readable until `:eof()` returns true. The `raw` argument is an optional string that needs to be equal to `'RAW'`, and will determine whether the data is compressed using the raw deflate format, or the zlib format. Any other string means the zlib format will be used.

9.5.4 Iso files

There is a fairly complete API for working with ISO files. Two related topics have their own pages: creating new images from scratch is covered in Building ISO images, and the patch generation mechanism behind the `:savePPF()` method below is covered in PPF patches.

- `PCSX.getCurrentIso()` will return an `Iso` object representing the currently loaded ISO file by the emulator.
- `PCSX.openIso(pathOrFile)` will return an `Iso` object opened from the specified argument, which can either be a filesystem path, or a `File` object.

The following methods are available on the `Iso` object:

```
:failed()       -- Returns true if the Iso file failed in some ways. The Iso object
is defunct if this is true.
:createReader() -- Returns an ISOReader object off the Iso file.
:open(lba[, size[, mode]]) -- Returns a File object off the specified span of
sectors.
:clearPPF()     -- Clears out all of the currently applied patches.
:savePPF()      -- Saves the currently applied patches to a PPF file named after the
ISO file.
```

The `:open` method has some magic built-in. The size argument is optional, and if missing, the code will attempt to guess the size of the underlying file within the Iso. It will represent the size of the virtual file in bytes. The size guessing mechanism can only work on MODE2 FORM1 or FORM2 sectors, and will result in a failed File object otherwise. The mode argument is optional, and can be one of the following:

- `'GUESS'`: will attempt to guess the mode of the file. This is the default.
- `'RAW'`: the returned File object will read 2352 bytes per sector.
- `'M1'`: the returned File object will read 2048 bytes per sector.
- `'M2_RAW'`: the returned File object will read 2336 bytes per sector. This can't be guessed. This is useful for extracting STR files that require the subheaders to be present.
- `'M2_FORM1'`: the returned File object will read 2048 bytes per sector.
- `'M2_FORM2'`: the returned File object will read 2324 bytes per sector.

The resulting File object will cache a single full sector in memory, meaning that small sequential reads won't read the same sector over and over from the disk.

The resulting File object will be writable, which will temporarily patch the CD-Rom image file in memory. It is possible to flush the patches to a PPF file by calling the `:savePPF()` method of the corresponding Iso object. When writing to one of these files, the filesystem metadata information will not be updated, meaning that the size of the file on the filesystem will not change, despite it being possible to write past the end of it and overflow on the next sectors. Note that while the virtual File object will enlarge to accommodate the writes, it will not be filled with zeroes as with typical filesystem operations, but instead will be filled with the existing data from the iso image. The written data is laid over the existing sector: only the part of the sector the mode exposes is replaced, and the rest, such as the sync header and location, is kept as it is

on the image. ECC and EDC will be recalculated on the fly for files opened in `'M2_FORM1'` or `'M2_FORM2'` mode, including when that mode was guessed. In any other mode, including `'M1'`, they are not recalculated.

The ISOReader object has the following methods:

```
1   :open(filename) -- Returns a File object off the specified file within the ISO.
```

This method is basically a helper over the `:open()` method of the Iso object, and will automatically guess the mode and size of the file.

9.6 Building ISO images

9.6.1 Introduction & Rationale

The `Iso` object described in the File API page works on an existing disc image. It is not read-only, since writing to it is what drives the PPF patch system, but what it can do is patch what is already there. The builder API is the other direction: it creates a fresh ISO9660 image from scratch, with a real filesystem in it, and writes it out to any `File` object.

The builder is filesystem-aware. You describe the directory tree you want, attach a `File` object as the content of each file, and the builder computes the whole layout at the end: the primary volume descriptor, both path tables, the directory extents, the LBA of every file, and the EDC/ECC of every sector. You never compute a sector address yourself unless you specifically want to.

Everything is deferred. `createRoot`, `createDir` and `createFile` only build an in-memory tree, and no byte is written until `close()` is called. This means the order in which you declare files does not have to match the order they end up on the disc, and it means a file's LBA is not known until after `close()`.

9.6.2 Creating a builder

```
1   PCSX.isoBuilder(file) -- returns an ISOBuilder object
```

The `file` argument is a `File` object opened for writing, which will receive the image. It can be a normal file on disk, or an in-memory buffer created with `Support.File.buffer()` if you only want the image transiently.

```
1   :failed() -- returns true if the output file is unusable
```

9.6.3 Writing the license sectors

```
1   :writeLicense(file) -- file is optional
```

The first 16 sectors of a PlayStation disc hold the license data. This is not a boot gate in the way it is often described: the license data *is* the boot logo, so what it mostly determines is what the console displays on the way in.

The `file` argument is a `File` object holding that data. It can be the official license file from the SDK, or simply a disc image dumped from another game, in which case the license is extracted from it. The two are told apart automatically, so you don't have to declare which one you are handing over.

Whether the license matters for booting depends on the machine:

- Japanese consoles refuse to boot a disc without a Japanese license.
- Very late European models refuse to boot a disc without a European license.
- Every other model boots regardless, though a missing or custom license changes the logo that comes up.
- Some emulators, unlike the hardware, refuse to boot without one at all.

A Japanese license is therefore the pragmatic choice, since it covers the widest range of consoles.

Calling `writeLicense()` with no argument, or with a file that failed to open, writes 16 sectors of zeroes instead. That is fine for development and for PCSX-Redux itself. No license data ships with PCSX-Redux, so you have to supply your own.

If you intend to write license sectors at all, do it before building the tree.

9.6.4 The volume descriptor

The PVD fields that identify the volume are settable, and are padded with spaces to their fixed ISO9660 widths for you:

```
:setSystemIdent(str)           :getSystemIdent()
:setVolumeIdent(str)           :getVolumeIdent()
:setVolSetIdent(str)           :getVolSetIdent()
:setPublisherIdent(str)        :getPublisherIdent()
:setDataPreparerIdent(str)     :getDataPreparerIdent()
:setApplicationIdent(str)      :getApplicationIdent()
:setCopyrightFileIdent(str)    :getCopyrightFileIdent()
:setAbstractFileIdent(str)     :getAbstractFileIdent()
:setBibliographicFileIdent(str) :getBibliographicFileIdent()
```

The getters return the stored string with its padding trimmed off.

For a disc meant to look like a retail PlayStation release, the system identifier is `PLAYSTATION`. The remaining fields are cosmetic as far as the console is concerned.

Everything else in the PVD is computed during `close()` and cannot be set: the volume space size, the path table sizes and locations, the root directory entry, the type code, the standard identifier, and both version fields.

9.6.5 Building the tree

```
:createRoot(sectorCount)               -- sectorCount defaults to 1
:createDir(parent, name, sectorCount)  -- sectorCount defaults to 1

:createFile(parent, name, contentFile)
```

All three return a `DirTree` node. `createRoot` must be called before anything else can be parented, and the node it returns is the root directory.

The `sectorCount` argument on the two directory functions is the number of sectors reserved for that directory's own extent. One sector holds roughly thirty entries depending on name lengths, so a directory with more children than that needs a larger count. It is not grown automatically.

`name` is the bare name. For files, the `;1` version suffix that ISO9660 requires is appended when the directory entry is serialized, so pass `SYSTEM.CNF`, not `SYSTEM.CNF;1`. Names are not validated or case-folded for you; ISO9660 level 1 wants uppercase 8.3, and it is on you to respect that if you care about the image being maximally portable.

`contentFile` is any `File` object. Its size at `close()` time is the size the file gets on the disc.

`createRoot` marks the root as having XA attributes, which is what a PlayStation disc looks like, and both `createDir` and `createFile` inherit the XA flag and the date from their parent. So setting a date on the root propagates to everything created under it afterwards, but not to anything created before.

9.6.6 DirTree nodes

The node returned by the three creation functions carries the per-entry knobs.

Read-only properties, meaningful after `close()` for the ones that depend on layout:

```
:getName()   :getSize()   :getLBA()   :isDir()
```

Navigation, all returning a node or `nil`:

```
:parent()   :firstChild()   :nextSibling()
```

Flags:

```
:isHidden()     :setHidden(bool)
:shouldSkip()   :setSkip(bool)
:hasXA()        :setXA(bool)
```

`setHidden` sets the ISO9660 hidden bit on the directory entry. The file is still fully present and readable by anything that ignores the bit.

`setSkip` is different and much stronger: the node still participates in the layout, so it is allocated an LBA and its content is written to the disc, but no directory entry is emitted for it. The file exists on the disc and is completely invisible to the filesystem. This is how you reproduce discs that address most of their data by hardcoded LBA and only expose a handful of real entries.

Sector mode and XA:

```
:setSectorMode(mode)   -- 'RAW', 'M1', 'M2_RAW', 'M2_FORM1', 'M2_FORM2'
:getXAAttribs()        :setXAAttribs(value)
:getXAFileNum()        :setXAFileNum(value)
```

Files default to `M2_FORM1`, which is what ordinary data on a PlayStation disc uses. `M2_FORM2` is for streaming data such as XA audio and STR video, where the sector carries 2324 bytes and no error correction. `M2_RAW` hands the builder full 2336-byte sectors including the subheader, for content that has to be reproduced byte for byte.

Dates:

```
:setDate(year, month, day, hour, minute, second, offset)
```

All seven arguments default to 0 if omitted. `year` is an offset from 1900 and `offset` is the timezone in 15-minute steps, matching the ISO9660 directory record format.

Layout overrides:

```
:hasAnchorLBA()      :getAnchorLBA()      :setAnchorLBA(lba)      :clearAnchorLBA()
:hasDeclaredSize()   :getDeclaredSize()   :setDeclaredSize(bytes) :clearDeclaredSize()
```

`setAnchorLBA` pins a node to a specific absolute LBA instead of letting the layout pass place it. Sectors between the previous end of the layout and the anchor are padded with empty M2_FORM1 data. `close()` throws if the layout cursor has already advanced past a requested anchor, since that cannot be satisfied.

`setDeclaredSize` overrides the `DataLength` field of the directory entry independently of how much content is actually attached. The pair is what lets you reproduce a disc whose single directory entry declares an extent spanning hundreds of megabytes of data that is itself addressed by LBA. Attach a small piece of content, declare the real extent, and place the rest of the data with `setSkip` nodes.

9.6.7 Closing

```
:close(threadCount) -- threadCount defaults to 0
```

This is where the work happens: the layout is computed, then the PVD, the volume descriptor set terminator, both path tables, the directory extents, the anchor padding and finally the file contents are written in that order.

EDC and ECC are computed for every sector the builder assembles itself, and the file contents are processed in parallel. `threadCount` is the number of worker threads to use for that; 0 means one per hardware thread. There is no way to turn the computation off, since an image with wrong EDC is not a useful image. The only exception is file content in `RAW` or `M2_RAW` mode, which already carries its own EDC and ECC and is written as supplied.

The builder does not add trailing padding past the end of the volume, and on real hardware that is a problem. The Mechacon has trouble seeking near the end of a disc: it regularly overshoots, and with nothing past the last real sector, it gets stuck there. Zero-filled sectors after the data are a landing zone that guarantees it cannot.

How many of them you need is a function of the read pattern. If the read is linear from the start of an extent and never goes near the edge of the written area, read-ahead slack is all there is to cover, and the 150 sectors `exe2iso` appends, two seconds of disc time, are right for a disc holding a single executable. If arbitrary seeks near the last written sector are possible, size against the overshoot instead: the authoring tool appends 9000, since an archive can be seeked into anywhere, including its own edge. Neither number is wrong; they answer different questions.

PCSX-Redux reads a truncated tail perfectly happily, so an image missing its padding passes every test you can run without a burner. Append the sectors after `close()`: the layout always starts at sector 16 and leaves the cursor past the end of the volume, so a loop of blank `writeSector` calls lands exactly where it should.

9.6.8 A complete example

```
local out = Support.File.open('mydisc.bin', 'TRUNCATE')
local builder = PCSX.isoBuilder(out)

builder:writeLicense(Support.File.open('license.dat'))
builder:setSystemIdent('PLAYSTATION')
builder:setVolumeIdent('MYDISC')

local root = builder:createRoot()

local cnf = Support.File.buffer()
cnf:write('BOOT=cdrom:\\PSX.EXE;1\r\nTCB=4\r\nEVENT=10\r\nSTACK=801FFFF0\r\n')
cnf:rSeek(0)
builder:createFile(root, 'SYSTEM.CNF', cnf)

local exe = builder:createFile(root, 'PSX.EXE', Support.File.open('mygame.ps-exe'))

local data = builder:createDir(root, 'DATA')
builder:createFile(data, 'LEVEL1.BIN', Support.File.open('level1.bin'))

local movie = builder:createFile(data, 'INTRO.STR', Support.File.open('intro.str'))
movie:setSectorMode('M2_FORM2')

builder:close()
print('PSX.EXE landed at LBA ' .. exe:getLBA())
out:close()
```

Note that `exe:getLBA()` is only meaningful after `close()`, since nothing is placed before then.

9.6.9 Reading the result back

The builder writes an image that the reader side can open directly, which makes round-tripping easy to verify without leaving Lua:

```
local buffer = Support.File.buffer()
local builder = PCSX.isoBuilder(buffer)
-- ...build...
builder:close()

buffer:rSeek(0)
local reader = PCSX.openIso(buffer):createReader()
local f = reader:open('DATA/LEVEL1.BIN;1')
print(f:size())
```

Note the `rSeek(0)` calls. A `File` object has separate read and write pointers, so a buffer you have just written content into still has its read pointer where you left it. The builder rewinds every content file itself before reading it during `close()`, so rewinding a buffer before handing it to `createFile` is only a precaution, but rewind the output before reopening it as an ISO.

This is exactly what the test suite in `tests/lua/isobuilder.lua` does.

9.6.10 Writing raw sectors

Everything above builds a filesystem. The builder can also write individual sectors directly, with no ISO9660 structure involved at all, which is what you want for reproducing a disc that does not have a conventional filesystem, or for placing data at a fixed location alongside one that does.

```
:writeSector(data[, mode])            -- writes at the current cursor, returns the
LBA used
:writeSectorAt(data, lba[, mode])     -- writes at an explicit LBA, returns it
:getCurrentLBA()                      -- where the cursor currently sits
```

`data` is a Lua string. A cdata pointer works too, but then the size has to be passed explicitly, since a pointer does not carry one:

```
:writeSector(ptr, size[, mode])
:writeSectorAt(ptr, size, lba[, mode])
```

`mode` defaults to `M2_FORM1`. Each mode consumes a different amount of data, and passing less than it needs is an error rather than a truncated sector:

mode	bytes consumed	notes
`M2_FORM1`	2048	ordinary data; sync, header, subheader and EDC/ECC are generated
`M2_FORM2`	2324	streaming data; EDC is generated
`M2_RAW`	2336	subheader onwards, supplied by you; sync and header are generated
`RAW`	2352	the complete frame, written verbatim

`M1` and `GUESS` cannot be written and will raise an error.

All LBAs here are relative to the start of the image, matching the rest of the ISO API, so sector 0 is the first sector of the file.

`writeSector` advances the cursor by one. `writeSectorAt` moves the cursor to just past the sector it wrote, but only ever forwards: writing behind the cursor leaves it where it was, so you can go back and fill something in without losing your place.

Note that raw sector writes and the filesystem API write to the same output file, and nothing arbitrates between them. `close()` always lays the filesystem out starting at sector 16, immediately after the system area, regardless of where the sector cursor happens to be, and will happily overwrite sectors you placed by hand. If you want to mix the two, use `setAnchorLBA` to push the filesystem past the region you are writing yourself.

The most useful thing this buys you is patching the image after the layout exists. A common shape on real discs is a lookup table that maps some game-side index to the LBA of the data it needs, which is a chicken-and-egg problem while you are building: the

table has to contain LBAs that nothing knows until the layout has been computed. Raw sector writes let you close that loop by going back afterwards.

```
local builder = PCSX.isoBuilder(out)
builder:writeLicense(license)
local root = builder:createRoot()

-- Reserve a sector for the table itself, plus the files it will point at.
local placeholder = Support.File.buffer()
placeholder:write(string.rep('\0', 2048))
placeholder:rSeek(0)
local table_ = builder:createFile(root, 'LBATABLE.BIN', placeholder)

local assets = {}
for i, name in ipairs(assetNames) do
    assets[i] = builder:createFile(root, name, Support.File.open(name))
end

-- Now the layout exists, and every node knows where it landed.
builder:close()

-- Build the real table from the computed LBAs and drop it over the placeholder.
local bit = require 'bit'
local sector = ''
for i, node in ipairs(assets) do
    local lba = node:getLBA()
    sector = sector .. string.char(bit.band(lba, 0xff), bit.band(bit.rshift(lba,
8), 0xff),
                                   bit.band(bit.rshift(lba, 16), 0xff),
bit.band(bit.rshift(lba, 24), 0xff))
end
sector = sector .. string.rep('\0', 2048 - #sector)
builder:writeSectorAt(sector, table_:getLBA(), 'M2_FORM1')
```

The EDC/ECC of the rewritten sector is recomputed as part of the write, so the patched image stays consistent.

9.7 PPF patches

9.7.1 Introduction & Rationale

PPF is the venerable patch format for disc images: a list of "at this absolute byte offset, replace these bytes" records, applied to a `.bin` image. PCSX-Redux both reads them and generates them, and the generation side is unusual enough to deserve an explanation, because there is no "make a patch" function anywhere. Patches are a side effect of writing to the disc.

The mental model is that the currently loaded disc image is writable in memory. Any write you perform is recorded as a difference against the original image, and at any point you can ask for the accumulated set of differences to be written out as a `.ppf` file. Nothing ever modifies the image on disk.

This makes the whole patch-authoring loop live: write to the disc from Lua, run the game right there to see the effect, adjust, and only save the PPF when the result is what you wanted.

9.7.2 Loading patches

Nothing needs to be done to apply an existing patch. When a disc image is opened, PCSX-Redux looks for a file next to it with the same name and a `.ppf` extension, and loads it automatically if present. When this happens, `[+ppf]` is appended to the image's description in the log.

Once loaded, patches are applied transparently on every sector read, so the emulator, the ISO browser, and the Lua reader API all see the patched disc.

The reader accepts PPF1, PPF2 and PPF3. A few things are worth knowing about the extra metadata the later versions carry:

- The version is taken from the binary version byte at offset 5, not from the digit in the ASCII magic. Those disagree in the wild, and the binary byte is the authoritative one.
- The `FILE_ID.DIZ` block of PPF2 and PPF3 files is read and kept. Note the length field is 32-bit in PPF2 and 16-bit in PPF3, not the other way around.
- The image size check of PPF2, and the 1024-byte verification block of PPF2 and PPF3, are skipped rather than checked. PCSX-Redux will happily apply a patch to an image it was not built for, so a patch that produces garbage is not going to be caught for you.
- PPF3 undo data is skipped. It is not retained and cannot be applied in reverse.
- Of the PPF3 image types, only type 0 (2352-byte `.bin`) is accepted. GI/PrimoDVD images are rejected.

9.7.3 Generating patches

Patches come from writing to the disc. Open a region of the current image as a `File` object, write to it, and every byte that ends up different from what the disc held before the write is recorded:

```
local iso = PCSX.getCurrentIso()
local f = iso:open(lba, size, mode)
f:writeAt('patched bytes', offset)
```

The details of `iso:open` and of what writing to a disc file does to sector headers and EDC/ECC are covered in the File API page. What matters here is what happens underneath: for each sector touched, the 2352-byte sector as it read before the write, with any existing patches applied, and the modified one are compared byte by byte, and every contiguous run of differing bytes becomes a patch record holding the new bytes. Writes that happen to store the value that was already there produce nothing.

The far more convenient way to reach a specific region is through the reader, which resolves a path to an LBA for you:

```
local iso = PCSX.getCurrentIso()
local f = iso:createReader():open('SLUS_012.34;1')
f:writeAt(newCode, 0x1234)
iso:savePPF()
```

Patches accumulate across writes. Writing to the same bytes repeatedly is fine: the records are collapsed before saving, so the final `.ppf` describes the end state rather than the history of how you got there.

9.7.4 Saving and clearing

```
:savePPF()  -- on an Iso object
:clearPPF() -- on an Iso object
```

`savePPF()` takes no filename. The destination is the disc image's own path with the extension replaced by `.ppf`, and an existing file at that path is truncated without asking. If you need the patch somewhere else, move it afterwards.

`clearPPF()` throws away every accumulated patch, returning the in-memory disc to the original image. Since the patches are the only record of your edits, this is not undoable.

Both are also reachable from the GUI, in the ISO browser, as the "Save PPF" and "Clear Patches" buttons, and over the web server at `api/v1/cd/ppf?function=save` and `?function=clear`.

9.7.5 What gets written

The generated file is always PPF1, regardless of what version was loaded. That is the version every PPF applier understands, and none of what the later versions add is data PCSX-Redux has: there is no undo information, no verification block, and no image size field.

The 50-byte description field and the `FILE_ID.DIZ` block are read from patches that carry them, but there is currently no way to set either one, from Lua or from anywhere else, so generated files carry the description of the patch that was loaded alongside the image, if any, and an empty description otherwise.

One consequence of PPF1 having a single-byte length field is that a long run of changed bytes is emitted as several consecutive records of at most 255 bytes each. This is normal and every applier handles it.

9.7.6 A worked example

Patching a binary on the disc, checking the result by running it, and only then saving:

```
local iso = PCSX.getCurrentIso()
local exe = iso:createReader():open('SLUS_012.34;1')

-- Replace an instruction with a nop.
exe:writeAt('\0\0\0\0', 0x8000)

-- The change is live on the disc now; reset and see what it does.
PCSX.resumeEmulator()

-- Happy with it:
iso:savePPF()
```

If it turns out not to be what you wanted, `iso:clearPPF()` puts the disc back and you can try again without ever having touched the image on disk.

9.8 Webserver Lua API

When the webserver is enabled, it will expose the `/api/v1/lua/` prefix, which can be used to execute Lua code on the emulator. When an endpoint with this prefix is called, the Lua table `PCSX.WebServer.Handlers` will be inspected to find a handler for the rest of the path in the endpoint. If a handler is found, it will be called with a request object representing the query, and it has to return a string, which will be sent back to the client as the response. If no handler is found, a 404 error will be returned. If an error occurs while executing the handler, a 500 error will be returned.

The request object has the following fields:

- `form` is a table of the part headers found in the request body. It is only filled if the content type of the request is `multipart/form-data`, and is empty otherwise. Each header name maps to an array of its values.
- `headers` is a table of the headers in the request. Each header name maps to an array of its values.
- `method` is the HTTP method of the request, as an uppercase string such as `POST`. Note that GET requests report `HTTP_GET`.
- `urlData` is a table with more information about the URL. It has the following string fields:
- `fragment`
- `host`
- `path`
- `port`
- `query`
- `schema`
- `userInfo`

If the returned string starts with the characters "HTTP/", then the web server will consider the response string is a full HTTP response with headers, and will send it as-is to the client. Otherwise, the response string will be sent as the body of a normal 200 response.

9.9 Memory and registers

9.9.1 FFI access

The Lua code can access the emulated memory and registers directly through some FFI bindings:

- `PCSX.getMemPtr()` will return a `cdata[uint8_t*]` representing up to 8MB of emulated memory. This can be written to, but careful about the emulated i-cache in case code is being written to.
- `PCSX.getParPtr()` will return a `cdata[uint8_t*]` representing up to 512kB of the EXP1/Parallel port memory space. This can be written to.
- `PCSX.getRomPtr()` will return a `cdata[uint8_t*]` representing up to 512kB of the BIOS memory space. This can be written to.
- `PCSX.getScratchPtr()` will return a `cdata[uint8_t*]` representing up to 1kB for the scratchpad memory space.
- `PCSX.getRegisters()` will return a structured cdata representing all the registers present in the CPU:
- `PCSX.getReadLUT()` will return a `cdata[uint8_t**]` representing the read LUT for the CPU.
- `PCSX.getWriteLUT()` will return a `cdata[uint8_t**]` representing the write LUT for the CPU.

```
typedef union {
    struct {
        uint32_t r0, at, v0, v1, a0, a1, a2, a3;
        uint32_t t0, t1, t2, t3, t4, t5, t6, t7;
        uint32_t s0, s1, s2, s3, s4, s5, s6, s7;
        uint32_t t8, t9, k0, k1, gp, sp, s8, ra;
        uint32_t lo, hi;
    } n;
    uint32_t r[34];
} psxGPRRegs;

typedef union {
    uint32_t r[32];
} psxCP0Regs;

typedef union {
    uint32_t r[32];
} psxCP2Data;

typedef union {
    uint32_t r[32];
} psxCP2Ctrl;

typedef struct {
    psxGPRRegs GPR;
    psxCP0Regs CP0;
    psxCP2Data CP2D;
    psxCP2Ctrl CP2C;
    uint32_t pc;
} psxRegisters;
```

9.9.2 Safer access

The above methods will return direct pointers into the emulated memory, so it's easy to crash the emulator if you're not careful. The `getMemoryAsFile()` method is safer, but will be slower:

- `PCSX.getMemoryAsFile()` will return a `File` object representing the full 4GB of accessible memory. All operations on this file will be translated to the emulated memory space. This is slower than the direct access methods, but safer. Any read or write operation will be clamped to the emulated memory space, and will not crash the emulator.

9.9.3 Memory mapping

PCSX-Redux will attempt to forward reads and writes for memory not mapped in the LUTs. This is useful for debugging, but will be slower than the direct access methods.

- `UnknownMemoryRead(address, size)` will be called when a read is attempted to an unmapped memory address. The function should return an 8, 16, or 32-bit value to be returned to the CPU.
- `UnknownMemoryWrite(address, size, value)` will be called when a write is attempted to an unmapped memory address. The function should return `true` or `false` indicating whether the write was handled.

9.10 Events

The Lua code can listen for events broadcasted from within the emulator. The following function is available to register a callback to be called when certain events happen:

```
1   PCSX.Events.createEventListener(eventName, callback)
```

Important: the return value of this function will be an object that represents the listener itself. If this object gets garbage collected, the corresponding listener will be removed. Thus it is important to store it somewhere that won't get garbage collected right away. The listener object has a `:remove` method to stop the listener before its garbage collection time.

The callback function will be called from an unsecured environment, and it is advised to delegate anything complex or risky enough to `PCSX.nextTick`.

The `eventName` argument is a string that can have the following values:

- `Quitting`: The emulator is about to quit. The callback will be called with an empty table. This is where you'd need to close libuv objects held by Lua through luv in order to allow the emulator to quit gracefully. Otherwise you may soft lock the application where it'll wait for libuv objects to close.
- `IsoMounted`: A new ISO file has been mounted into the virtual CDRom drive. The callback will be called with an empty table.
- `GPU::Vsync`: The emulated GPU has just completed a vertical blanking interval. The callback will be called with an empty table.
- `ExecutionFlow::ShellReached`: The emulation execution has reached the beginning of the BIOS' shell. The callback will be called with an empty table. This is the moment where the kernel is properly set up and ready to execute any arbitrary binary. The emulator may use this event to side load binaries, or signal gdb that the kernel is ready.
- `ExecutionFlow::Run`: The emulator resumed execution. The callback will be called with an empty table. This event will fire when calling `PCSX.resumeEmulator()`, when the user presses Start, or other potential interactions.
- `ExecutionFlow::Pause`: The emulator paused execution. The callback will be called with a table that contains a boolean named `exception`, indicating if the pause is the result of an execution exception within the emulated CPU. This event will fire on breakpoints too, so if breakpoints have Lua callbacks attached on them, they will be executed too.
- `ExecutionFlow::Reset`: The emulator is resetting the emulated machine. The callback will be called with a table that contains a boolean named `hard`, indicating if the reset is a hard reset or a soft reset. This event will fire when calling `PCSX.softResetEmulator()` or `PCSX.hardResetEmulator()`, when the user presses Reset, or other potential interactions.
- `ExecutionFlow::SaveStateLoaded`: The emulator just loaded a savestate. The callback will be called with an empty table. This event will fire when calling `PCSX.loadSaveState()`, when the user loads a savestate, or other potential interactions. This is useful to listen to in case some internal state needs to be reset within the Lua logic.
- `GUI::JumpToPC`: The UI is being asked to move the assembly view cursor to the specified address. The callback will be called with a table that contains a number named `pc`, indicating the address to jump to.

- `GUI::JumpToMemory`: The UI is being asked to move the memory view cursor to the specified address. The callback will be called with a table that contains a number named `address`, indicating the address to jump to, and `size`, indicating the number of bytes to highlight.
- `Keyboard`: The emulator is dispatching keyboard events. The callback will be called with a table containing four numbers: `key`, `scancode`, `action`, and `mods`. They are the same values as the glfw callback set by `glfwSetKeyCallback`.
- `Memory::SetLuts`: The emulator has updated the memory LUTs. The callback will be called with an empty table.

9.11 Breakpoints

If the debugger is activated, and while using the interpreter, the Lua code can insert powerful breakpoints using the following API:

Breakpoints added while the debugger is off are accepted, but won't trigger until it is turned on, either with the `-debugger` command line flag or from the configuration menu. `PCSX.addBreakpoint` prints a warning in that case.

```
PCSX.addBreakpoint(address, type, width, cause, invoker)
```

Important: the return value of this function will be an object that represents the breakpoint itself. If this object gets garbage collected, the corresponding breakpoint will be removed. Thus it is important to store it somewhere that won't get garbage collected right away.

The only mandatory argument is `address`, which will by default place an execution breakpoint at the corresponding address. The second argument `type` is an enum which can be represented by one of the 3 following strings: `'Exec'`, `'Read'`, `'Write'`, and will set the breakpoint type accordingly. The third argument `width` is the width of the breakpoint, which indicates how many bytes should intersect from the base address with operations done by the emulated CPU in order to actually trigger the breakpoint. The fourth argument `cause` is a string that will be displayed in the logs about why the breakpoint triggered. It will also be displayed in the Breakpoint Debug UI. And the fifth and final argument `invoker` is a Lua function that will be called whenever the breakpoint is triggered. By default, this will simply call `PCSX.pauseEmulator()`. If the invoker returns `false`, the breakpoint will be permanently removed, permitting temporary breakpoints for example. The signature of the invoker callback is:

```
function(address, width, cause)
    -- body
end
```

The `address` parameter will contain the address that triggered the breakpoint. For `'Exec'` breakpoints, this is going to be the same as the current `pc`, but for `'Read'` and `'Write'`, it's going to be the actual accessed address. The `width` parameter will contain the width of the access that triggered the breakpoint, which can be different

from what the breakpoint is monitoring. And the `cause` parameter will contain a string describing the reason for the breakpoint; the latter may or may not be the same as what was passed to the `addBreakpoint` function. Note that you don't need to strictly adhere to the signature, and have zero, one, two, or three arguments for your invoker callback. The return value of the invoker callback is also optional.

For example, these two examples are well formed and perfectly valid:

```
bp1 = PCSX.addBreakpoint(0x80000000, 'Write', 0x80000, 'Write tracing',
function(address, width, cause)
    local regs = PCSX.getRegisters()
    local pc = regs.pc
    print('Writing at ' .. address .. ' from ' .. pc .. ' with width ' .. width ..
' and cause ' .. cause)
end)

bp2 = PCSX.addBreakpoint(0x80030000, 'Exec', 4, 'Shell reached - pausing',
function()
    PCSX.pauseEmulator()
    return false
end)
```

The returned breakpoint object will have a few methods attached to it:

- `:disable()`
- `:enable()`
- `:isEnabled()`
- `:remove()`

A removed breakpoint will no longer have any effect whatsoever, and none of its methods will do anything. Remember it is possible for the user to still manually remove a breakpoint from the UI.

Note that the breakpoint will run outside of any safe Lua environment, so it's possible to crash the emulator by doing something wrong which would normally be caught by the safe environment of the main thread. This is to ensure that the breakpoint can run as

fast as possible. In order to avoid this, it's possible to wrap the invoker callback in a `pcall` call, which will catch any error and display it in the logs. For example:

```
local someActualFunction = function(address, width, cause)
    -- body
end
bp = PCSX.addBreakpoint(0x80030000, 'Write', 4, 'Shell write tracing',
function(address, width, cause)
    local success, msg = pcall(function()
        someActualFunction(address, width, cause)
    end)
    if not success then
        print('Error while running Lua breakpoint callback: ' .. msg)
    end
end)
```

This will ensure that the breakpoint will never crash the emulator, and will instead display the error in the logs, but it will also slow down the execution of the breakpoint. It's up to the user to decide whether or not this is acceptable.

It is safe to add or remove breakpoints from within a breakpoint callback, but it's not safe to remove the breakpoint that is currently being executed. For this specific case, simply return `false` from the invoker callback, and the breakpoint will be removed after the callback returns.

9.12 Inline assembler

There is a Lua API for an inline MIPS assembler.

One can instantiate an assembler with `PCSX.Assembler.New()`, which will keep all the state of the assembler. The assembler can be used to assemble a string of MIPS code, and then compile it to memory or a file.

The object has the following methods:

- `:parse(code)` will parse the string `code` and assemble it. It will return the assembler object itself, so it can be chained with the compile methods. The parser is fairly simple, but it should be enough for most cases. The parser should handle all of the basic MIPS instructions, all of the PS1's GTE opcodes, and many pseudo-instructions. Labels can be used as operands, and will be resolved against the symbols known to the emulator, but defining labels within the assembled code isn't supported. The parser is more lenient than normal MIPS assemblers, and will accept some invalid syntax, but it will throw an error if it can't parse the code.
- `:compileToUint32Table(baseAddress)` will compile the assembled code to a table of `uint32_t` values. This is useful for debugging, but not very useful for actually running the code. The `baseAddress` is the address that the code will be loaded at, in order to handle relative jumps.
- `:compileToMemory(memory, baseAddress, memoryStartAddress)` will compile the assembled code to an indexable memory object, such as an ffi array. The memory object must be at least as large as the assembled code. The memory object will be modified in-place. The `baseAddress` is the address that the code will be loaded at, in order to handle relative jumps. The `memoryStartAddress` is the address that the memory object starts at.
- `:compileToFile(file, baseAddress, fileStartAddress)` will compile the assembled code to a file object. The file object must be at least as large as the assembled code. The file object will be modified in-place. The `baseAddress` is the address that the code will be loaded at, in order to handle relative jumps. The `fileStartAddress` is an optional argument which defaults to 0, and is the address that the file object starts at. Using a 0-based file address is relevant when using with the `PCSX.getMemoryAsFile()` function, or when using a `Support.mem4g()` File object.

9.13 Handling of PSX binaries

There is some support for handling PSX binaries in the Lua API. The `PCSX.Binary` module has the following functions:

- `PCSX.Binary.load(input, output)`: loads an input `File` object into an output `File` object. The input file must be a valid PSX binary, which can be in the formats CPE, PS-EXE, PSF, or ELF, and the output file must be at least 4GB large, which means it's really only suitable with the `mem4g` object, or the object returned by `getMemoryAsFile()`. The output file will be modified in-place. The output file will be loaded at the address specified in the binary header. If successful, the function will return an info structure with the following optional fields:
- `pc`: the entry point of the binary
- `gp`: the global pointer of the binary
- `sp`: the stack pointer of the binary
- `region`: the region of the binary, which can be one of the following:
- `'NTSC'`: NTSC region
- `'PAL'`: PAL region
- `PCSX.Binary.pack(src, dest, addr, pc, gp, sp, options)`: compresses the input binary stream into a self-decompressing stream. The input must be a `File` object, and the output must be a `File` object. The `addr` is the address that the binary will be loaded at. The `pc`, `gp`, and `sp` are the entry point, global pointer, and

stack pointer of the binary. The `options` is an optional table with the following optional fields:

- `tload`: the address that the compressed binary will be loaded at. If not specified, it will be set to a suitable address. Not specifying this will generate an in-place decompression binary, which doesn't require much extra memory. When specifying this, the whole output stream will be loaded at this specific address, and the decompression code will be located at its beginning, meaning both the entry point and the loading addresses will be the same.
- `nopad`: the generated PS-EXE will not be padded to 2048 bytes. It will not be suitable to boot from cd-rom, as the BIOS requires binaries to be aligned to sector sizes, but many other tools like unirom+nops or caetla+catflap will be able to handle it properly.
- `booty`: a boolean specifying that the output stream will be suitable to boot as a PIO bytestream. Incompatible with `tload` or `raw`.
- `nokernel`: a boolean specifying that the produced binary will not try to call into the kernel, in case the kernel has been wiped out. Results in a slightly bigger binary, but is necessary when the retail kernel is not present.
- `shell`: a boolean specifying that the output stream will attempt to reboot the machine and load the binary, which can be useful when resetting the kernel.
- `raw`: a boolean specifying that the output stream will be a raw binary, without a PS-EXE header. The generated binary will be completely position independent, and will not require any special loading address. It is up to the user to ensure no overlap can happen by loading the file to a high enough address. This option can be used to generate embedded binaries within others, or to be loaded by other means, and executed by jumping to it. The `tload` option will be ignored when this is specified.
- `rom`: a boolean specifying that the output stream will be a ROM file suitable to be flashed on a cheat cartridge, as long as the cartridge itself has linear addressing, which is not necessarily the case for all cartridges. The `tload` option will be ignored when this is specified.
- `cpe`: a boolean specifying that the output stream will be a CPE file, which is the file format used by the ancient toolchain by Sony. This can be useful when trying to load binaries with these ancient tools.
- `PCSX.Binary.createExe(src, dest, addr, pc, gp, sp)`: creates a PS-EXE binary from the input binary stream. The input must be a `File` object, and the output must

be a `File` object. The `addr` is the address that the binary will be loaded at. The `pc`, `gp`, and `sp` are the entry point, global pointer, and stack pointer of the binary.

The above methods can be used for example the following way:

```
local src = PCSX.getCurrentIso():createReader():open('SLUS_012.34;1')

local m4g = Support.File.mem4g()
local info = PCSX.Binary.load(src, m4g)
local asm = PCSX.Assembler.New()
asm:parse [[
    lui   $a0, 0x8001
    addiu $a0, 0x1234
]]:compileToFile(m4g, 0x80045678)
local bytes = m4g:subFile(m4g:lowestAddress(), m4g:actualSize())

local dst = Support.File.open('compressed-from-lua.ps-exe', 'TRUNCATE')

PCSX.Binary.pack(bytes, dst, m4g:lowestAddress(), info.pc, info.gp, info.sp)
```

Additionally, the `PCSX.Misc` module has the following functions:

- `PCSX.Misc.uclPack(src, dest)`: compresses the input binary stream into a ucl-compressed stream. The input can be a `File` object, a string, a LuaBuffer, or a `Slice`. The output is optional, and can be a `File` object, a LuaBuffer, or a `Slice`. If the output is omitted, the function returns a new `Slice` with the compressed data, otherwise it returns the compressed size. A `File` output will be written at its current write pointer, and will be compressed using the UCL-NRV2E compression algorithm, which is a variant of the UCL compression algorithm. The output stream can be decompressed in-place with very little memory overhead. Simply place the compressed data at the end of the decompression buffer + 16 bytes. The stream doesn't require to be aligned in any particular way.
- `PCSX.Misc.writeUclDecomp(dest)`: writes a MIPS UCL-NRV2E decompression routine to the output `File` object, at its current write pointer. The function returns the number of bytes written, which at the moment is 340 bytes. The code is position independent, and has the following function signature:
- `void decompress(const uint8_t* src, uint8_t* dest);`

9.14 Case studies

9.14.1 Spyro: Year of the Dragon

By looking up some of the gameshark codes for this game, we can determine the following memory addresses:

- `0x8007582c` is the number of lives.
- `0x80078bbc` is the health of Spyro.
- `0x80075860` is the number of unspent jewels available to the player.
- `0x80075750` is the number of dragons Spyro released so far.

With this, we can build a small UI to visualize and manipulate these values in real time:

```lua
-- Declare a helper function with the following arguments:
--   mem: the ffi object representing the base pointer into the main RAM
--   address: the address of the uint32_t to monitor and mutate
--   name: the label to display in the UI
--   min, max: the minimum and maximum values of the slider
--
-- This function is local as to not pollute the global namespace.
local function doSliderInt(mem, address, name, min, max)
  -- Clamping the address to the actual memory space, essentially
  -- removing the upper bank address using a bitmask. The result
  -- will still be a normal 32-bits value.
  address = bit.band(address, 0x1fffff)
  -- Computing the FFI pointer to the actual uint32_t location.
  -- The result will be a new FFI pointer, directly into the emulator's
  -- memory space, hopefully within normal accessible bounds. The
  -- resulting type will be a cdata[uint8_t*].
  local pointer = mem + address
  -- Casting this pointer to a proper uint32_t pointer.
  pointer = ffi.cast('uint32_t*', pointer)
  -- Reading the value in memory
  local value = pointer[0]
  -- Drawing the ImGui slider
  local changed
  changed, value = imgui.SliderInt(name, value, min, max, '%d')
  -- The ImGui Lua binding will first return a boolean indicating
  -- if the user moved the slider. The second return value will be
  -- the new value of the slider if it changed. Therefore we can
  -- reassign the pointer accordingly.
  if changed then pointer[0] = value end
end

-- Utilizing the DrawImguiFrame periodic function to draw our UI.
-- We are declaring this function global so the emulator can
-- properly call it periodically.
function DrawImguiFrame()
  -- This is typical ImGui paradigm to display a window using
  -- the safe mode. This will ensure that the window is properly
  -- closed even if an exception is thrown during the rendering
  -- of the window.
  imgui.safe.Begin('Spyro internals', function()
    -- Grabbing the pointer to the main RAM, to avoid calling
    -- the function for every pointer we want to change.
    -- Note: it's not a good idea to hold onto this value between
    -- calls to the Lua VM, as the memory can potentially move
    -- within the emulator's memory space.
    local mem = PCSX.getMemPtr()

    -- Now calling our helper function for each of our pointer.
    doSliderInt(mem, 0x8007582c, 'Lives', 0, 9)
    doSliderInt(mem, 0x80078bbc, 'Health', -1, 3)
    doSliderInt(mem, 0x80075860, 'Jewels', 0, 65000)
    doSliderInt(mem, 0x80075750, 'Dragons', 0, 70)
```